Introduction to R and R Studio (Posit)



Pushpakanthie Wijekoon

Introduction to R and RStudio (Posit)

Pushpakanthie Wijekoon

Introduction to R and RStudio (Posit)

Copyright © 2023 Pushpakanthie Wijekoon
All rights reserved.

Preface

Welcome to "Introduction to R and R Studio: Unlocking the Power of Data Analysis." In this book, you will be guided through the fundamental concepts of R and R Studio, as well as the utilization of these tools for obtaining descriptive summary statistics and creating compelling visualizations.

In today's data-driven world, the ability to extract meaningful insights from raw data is highly valued. As the field of research continues to evolve and generate vast amounts of data, the need for effective data analysis tools becomes increasingly crucial. In the ever-evolving landscape of data analysis and statistical computing, R, a versatile programming language, and R Studio, an intuitive integrated development environment (IDE), has emerged as a powerful tool for researchers across various disciplines. With its vast array of packages and libraries, R provides a flexible and efficient platform for data manipulation, visualization, and statistical modeling.

This book has been specifically designed to introduce you to the world of R and R Studio, providing a solid foundation for data analysis and visualization. Starting from the basics, you will be guided through each step, ensuring a comprehensive understanding of the core concepts and proficiency in utilizing R and R Studio for analytical tasks. Whether you are a researcher, student, or professional in the field, this book will serve as your compass in navigating the fundamentals of R programming.

The chapters in this book cover a range of topics, including data import and manipulation, descriptive statistics, and exploratory data analysis. Various visualization methods will be explored, enabling the effective communication of findings and the identification of patterns, trends, and outliers within your data.

Throughout the book, clear explanations, illustrative examples, and hands-on exercises are provided to reinforce your understanding. The objective is to equip you with the necessary knowledge and skills to confidently analyze data, generate descriptive summary statistics, and create visually engaging representations using R and R Studio.

Active engagement with the content, application of techniques to your own datasets, and experimentation with different visualization approaches are strongly encouraged. The practical experience gained through these exercises will enhance your proficiency and enable you to tackle real-world data analysis challenges.

It is my hope that "Introduction to R and R Studio" becomes your go-to resource for mastering data analysis and visualization, empowering you to unlock valuable insights, make informed decisions, and excel in your data-driven pursuits.

This book is used as a guide for the participants attending the workshop held in the National Institute of Fundamental Studies on the Introduction to R and R Studio (Posit).

Pushpakanthie Wijekoon
Emeritus Professor,
Department of Statistics & Computer Science,
University of Peradeniya.

Table of Contents

Preface i
1. Introduction to R and RStudio (Posit) 1
1.1 What is R? 1
1.2 Installing R on Windows 1
1.2.1 Using R commands 3
1.3 The RStudio (Posit) Development Environment 4
1.3.1 Installing RStudio in Windows 4
1.4 R Studio (Posit) Cloud 5
1.5 Using Projects 6
1.5.1 Creating Projects 6
1.5.2 Opening Projects 6
1.5.3 Quitting a Project 7
2. Types of Data Objects in R 8
2.1 Writing variable names 8
2.2 Check your data and help function in the relevant R package 8
2.3 Data Types 9
2.4 Importing data 11
2.5 Exporting data 13
3. R Packages 14
3.1 What is a package? 14
3.2 Installing packages in RStudio. 14
3.3 Loading Packages 16
4. R Markdown, R Notebook and Quarto 17
4.1 What is R Markdown? 17
4.2 What is R Notebook? 17
4.3 What is Quarto documents? 21
5. Descriptive Summary Statistics 23
5.1 Two Branches of Statistics: 23
5.2 Types of Data 24
5.3 Descriptive Statistics 24
5.3.1 Measures of Central Tendencies 24
5.3.2 Measures of Relative Position 25

5.3.3 Measures of Variation 26
5.3.4 Measures of Shapes 27
5.4 Normality tests 27
6. Data Visualization 30
6.1 Univariate graphs 30
6.1.1 Univariate graphs for categorical data 30
6.1.2 Univariate graphs for quantitative data 34
6.2 Bivariate Graphs 39
6.2.1 Categorical vs. Categorical graphs 39
6.2.2. Quantitative vs. Quantitative 43
6.2.3 Categorical vs. Quantitative 44
6.3 Multivariate graphs 49
6.3.1 Drawing a multivariate scatter plot 49
6.3.2 Drawing histograms for multivariate data 50
6.3.3 Mean/SE plots for multivariate data 51
6.3.4 Bubble chart for multivariate data 52
6.3.5 Correlation plots 53
6.3.6 Biplots 55
6.3.7 Scatterplot matrix 57
6.3.8 Heatmap 58
6.3.9 Interactive Graphs 59
6.4 Drawing graphs using Plotly 60
6.5 Some other graphing facilities 61
References 62

1. Introduction to R and RStudio (Posit)

1.1 What is R?

R is an open-source environment for statistical computing and visualization. It is based on the S language developed at Bell Laboratories in the 1980's. There are no restrictions on access or use - *It's free!*

R is an integrated suite of software facilities for data manipulation, calculation and graphical display. It includes

- an effective data handling and storage facility,
- a suite of operators for calculations on arrays, in particular matrices,
- a large, coherent, integrated collection of intermediate tools for data analysis,
- graphical facilities for data analysis and display either on-screen or on hardcopy, and
- a well-developed, simple and effective programming language which includes conditionals, loops, user-defined recursive functions and input and output facilities.

Practicing statisticians have implemented thousands of specialized statistical procures for a wide variety of applications as contributed packages, which are also freely available and which integrate directly into R. The main source of information about the R system is the World Wide Web with the official home page of the R project being www.R-project.org . Most resources are available from this page: the R system itself, a collection of add-on packages, manuals, documentation and more.

Advantages and Benefits of Using R

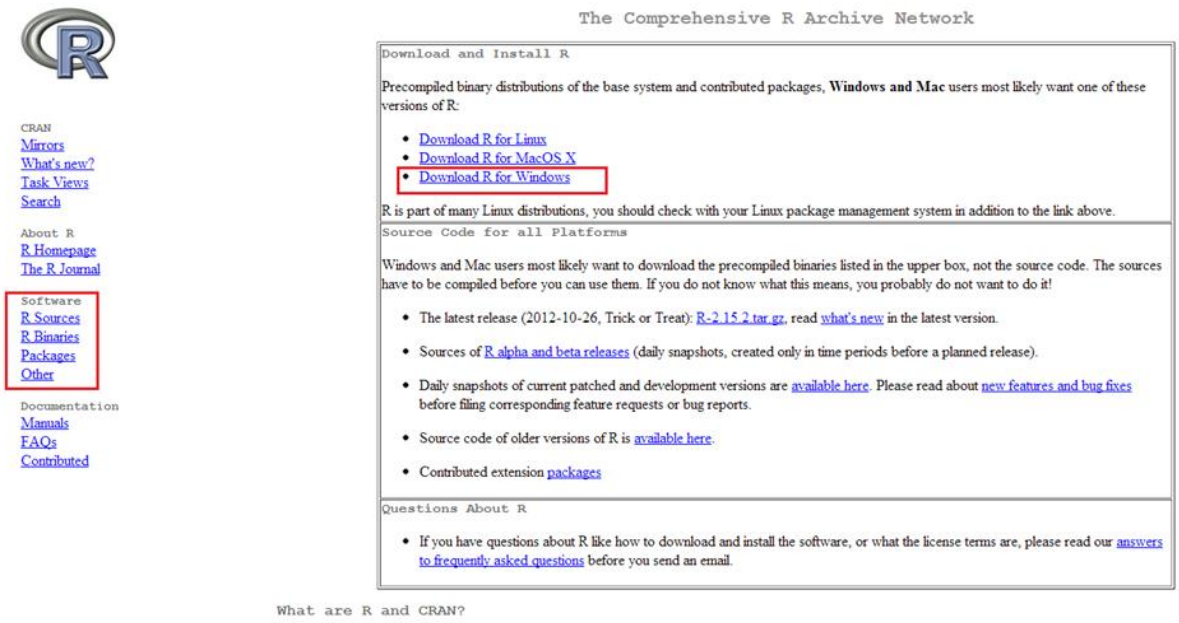
- is free and open source
- runs on Windows, Linux, Mac
- virtually does any statistical analysis
- does wonderful graphs
- is on the cutting edge of statistical tools development

Possible drawbacks

- has a steep learning curve (the beginning is a little hard)
- is austere (originally no drop-down menu existed, but things have changed, as we are going to see).

1.2 Installing R on Windows

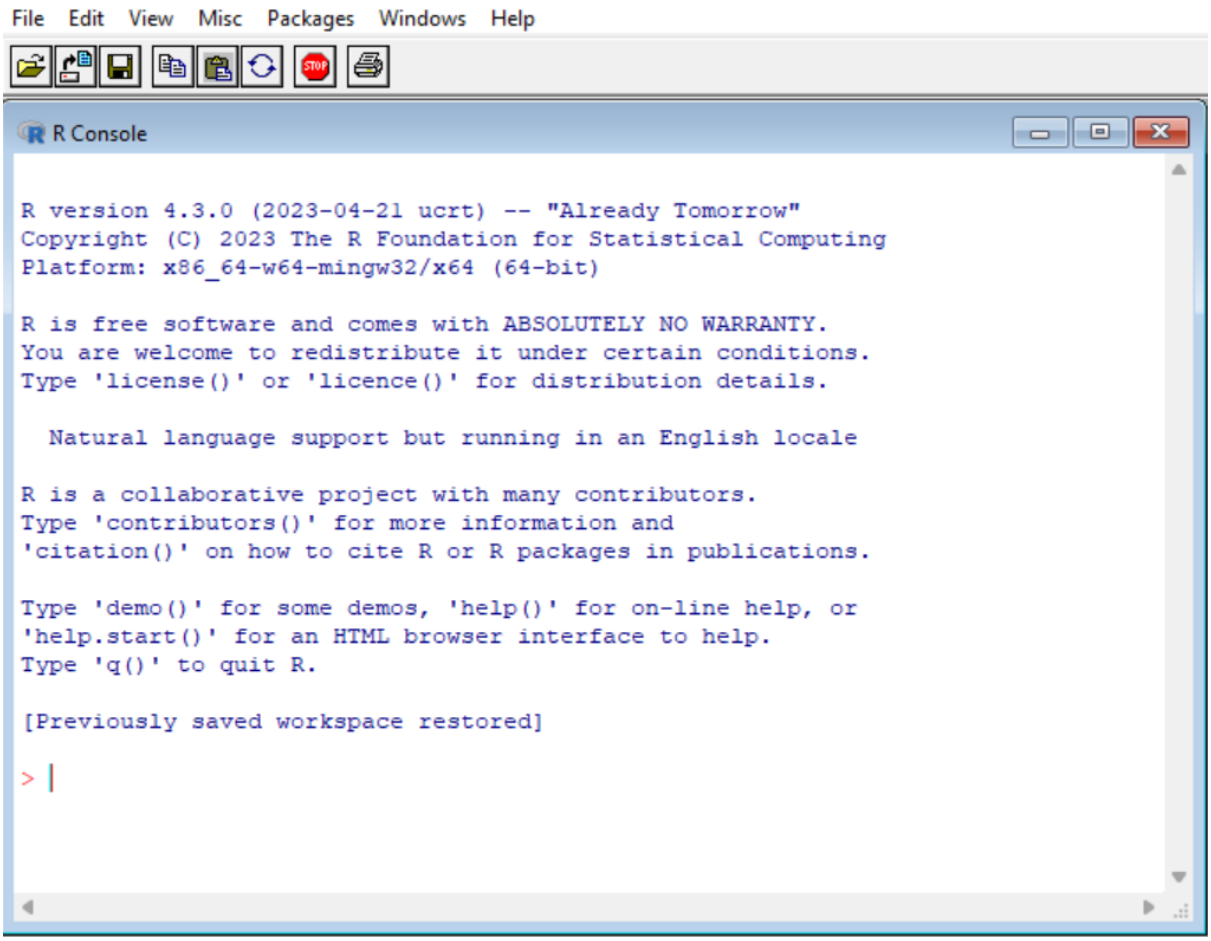
You can download and install the current Windows version of R from www.cran.r-project.org CRAN (Comprehensive R Network) website. New releases of R are made very regularly as R is actively being improved all the time. It is worthwhile installing new versions of R regularly, to make sure that you have a recent version of R (to ensure compatibility with all the latest versions of the R packages that you have downloaded).



1. Under “Download and Install R”, click on the “Download R for Windows” link.
2. Under “Subdirectories”, click on the “base” link.
3. On the next page, you should see a link saying something like “Download R 4.3.0 for Windows” (or R X.X.X, where X.X.X gives the version of R,). Click on this link.
4. Download and save the file on the Desktop. Then double-click on the icon for the file to run it.
5. You will be asked what language to install it in - choose English.
6. The R Setup Wizard will appear in a window. Click “Next” at the bottom of the R Setup wizard window. Click “Next” again……
7. R should now be installed. This will take about a minute. When R has finished, you will see “Completing the R for Windows Setup Wizard” appear. Click “Finish”.

After installing R,

- Check if there is an “R” icon on the desktop of the computer that you are using. If so, double-click on the “R” icon to start R.
- If you cannot find an “R” icon, click on the “Start” button at the bottom left of your computer screen, and then choose “All programs”, and start R by selecting “R” (or R X.X.X, where X.X.X gives the version of R) from the menu of programs.
- The R console (a rectangle) should pop up.



1.2.1 Using R commands

You will type R commands into the R console in order to carry out analyses in R. In the R console, you will see:

```
>
```

This is the R prompt. We type the commands needed for a particular task after this prompt. The command is carried out after you hit the Return key.

Once you have started R, you can start typing in commands, and the results will be calculated immediately, for example:

```
> (10*2)+(5*2)
[1] 30
```

All variables (scalars, vectors, matrices, etc.) created by R are called objects. In R, we assign values to variables using an arrow. For example, we can assign the value 5*6 to the variable x using the command:

```
> x <- 5*6
```

To view the contents of any R object, just type its name and the contents of that R object will be displayed:

```
> x
[1] 30
```

There are several ways to work with R:

- with the R console GUI
- with the RStudio IDE
- with Rcmdr add-on package

1.3 The RStudio (Posit) Development Environment

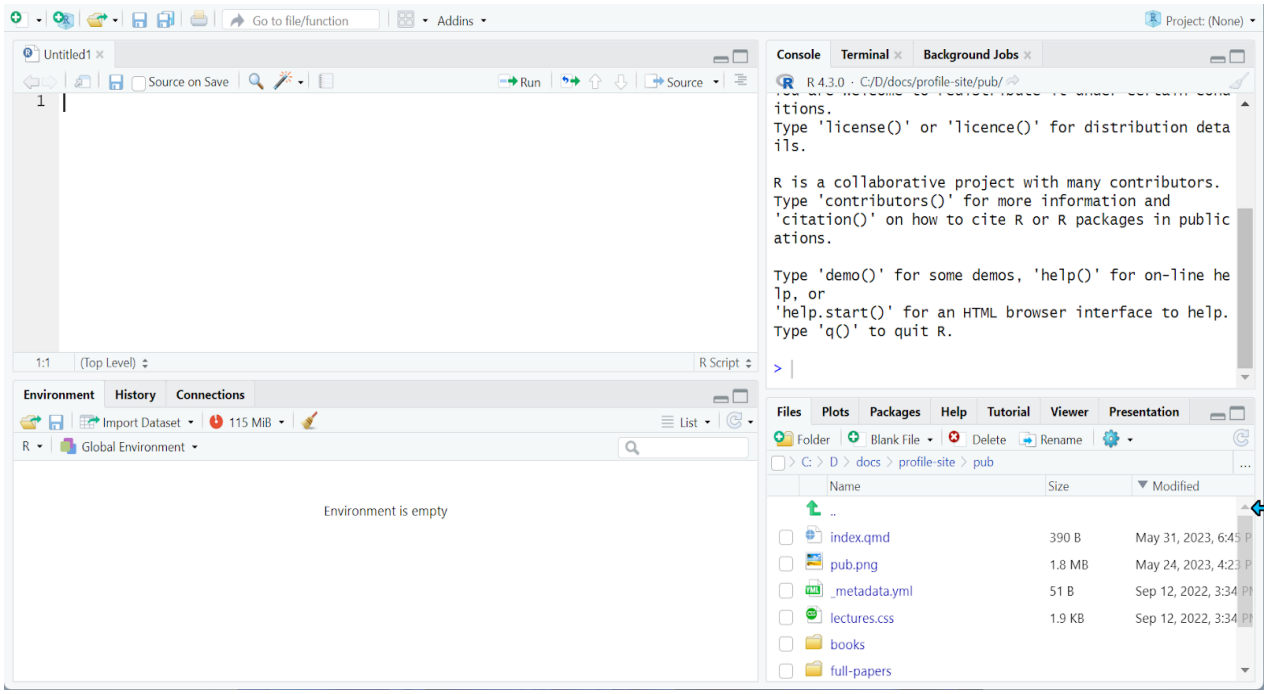
RStudio is an excellent cross-platform integrated development environment (IDE) for R. This environment includes the command line interface, a code editor, output graphs, history, help, workspace contents, and a package manager all in one attractive interface.

In 2022, R Studio was renamed as Posit to widen the programming environment to focus and start looking at how Python can be improved for Data Scientists.

1.3.1 Installing RStudio in Windows

1. Download RStudio from http://rstudio.org/download/desktop and install it. Leave all default settings in the installation options.
2. Open RStudio.

When you first launch RStudio, you can see an interface that looks similar to this:



The RStudio interface consists of several windows.

1. **Top right :** console window (also called command window). Here you can type simple commands after the ">" prompt and R will then execute your command. This is the most important Window since this is where the R codes can be run.
2. **Bottom right:** files / plots / packages / help window. Here you can open files, view plots (also previous plots), install and load packages or use the help function.
3. **Top left:** editor window (also called script window). Collections of commands (scripts) can be edited and saved. When you don't get this window, you can open it with **File → New → R script.** Just typing a command in the editor window is not enough; it has to get into the command window before R executes the command. If you want to run a line from the script window (or the whole script), you can click Run or press CTRL+ENTER to send it to the command window.
4. **Bottom left:** workspace / history window. In the workspace window you can see which data and values R has in its memory. You can edit the values by clicking on them. The history window shows what has been typed before.

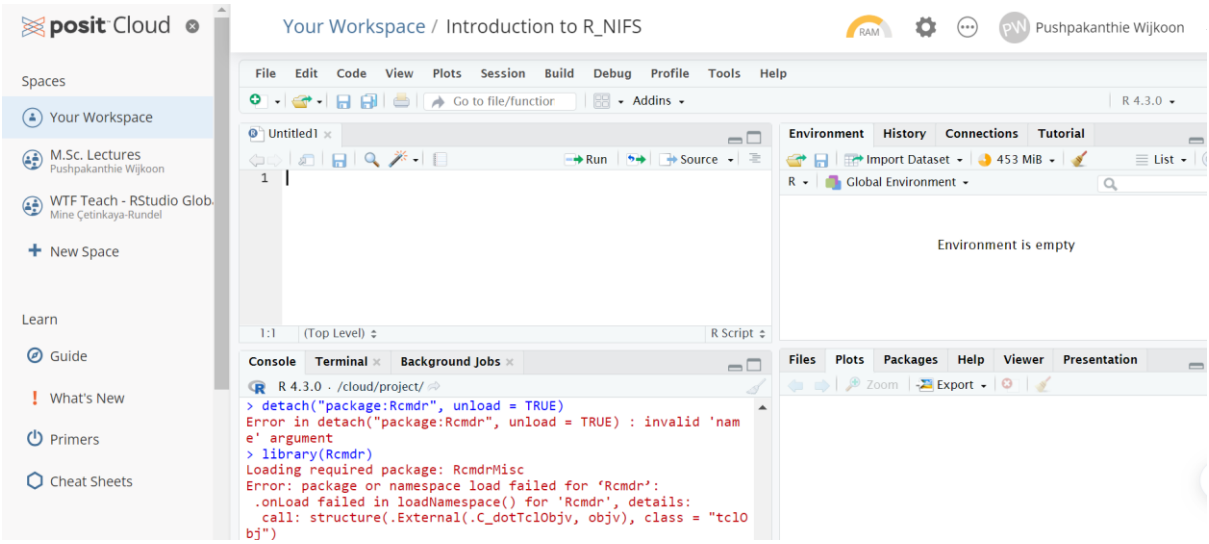
You can change the size of the windows by dragging the grey bars between the windows. You can also change the positions of these windows by selecting,

Tools—-> Global Options ——-> Pane Layout.

1.4 R Studio (Posit) Cloud

You can also work on R without installing R or R Studio in your computer. There is a cloud base working environment for R called Posit Cloud (previously RStudio Cloud), which enables you to access Posit's robust collection of data science tools directly from your browser without the need for installation or complicated configuration.

To use it browse the link https://posit.cloud, and create a free account.



1.5 Using Projects

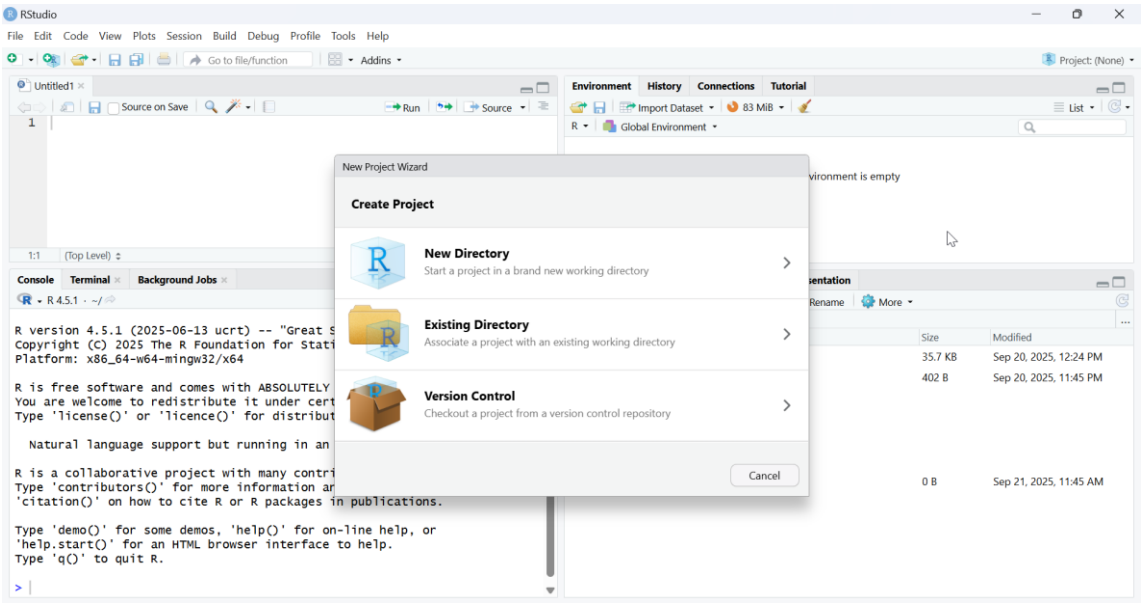
RStudio projects make it straightforward to divide your work into multiple contexts, each with its own working directory, workspace, history, and source documents.

1.5.1 Creating Projects

RStudio projects are associated with R working directories. You can create an RStudio project:

- in a new directory, or
- in an existing directory where you already have R code and data

To create a new project, use the **New Project** command (available on the File menu and on the global toolbar):



When a new project is created in RStudio:

1. Creates a project file (with an .Rproj extension) within the project directory. This file contains various project options (discussed below) and can also be used as a shortcut for opening the project directly from the file system.
2. Loads the project into RStudio and display its name in the Projects toolbar (which is located on the far right side of the main toolbar).

1.5.2 Opening Projects

There are several ways to open a project:

- Using the **Open Project** command (available from both the Projects menu and the Projects toolbar) to browse for and select an existing project file (e.g. MyProject.Rproj).
- Selecting a project from the list of most recently opened projects (also available from both the Projects menu and toolbar).
- Double-clicking on the project file within the system

When a project is opened within RStudio the following actions are taken:

1. A new R session (process) is started
2. The .Rprofile file in the project's main directory (if any) is sourced by R
3. The .RData file in the project's main directory is loaded (if project options indicate that it should be loaded).
4. The .Rhistory file in the project's main directory is loaded into the RStudio History pane (and used for Console Up/Down arrow command history).
5. The current working directory is set to the project directory.
6. Previously edited source documents are restored into editor tabs
7. Other RStudio settings (e.g. active tabs, splitter positions, etc.) are restored to where they were the last time the project was closed.

1.5.3 Quitting a Project

When you are within a project and choose to either Quit, close the project, or open another project, the following actions are taken:

- .RData and/or .Rhistory are written to the project directory (if current options indicate they should be)
- The list of open source documents is saved (so it can be restored next time the project is opened)
- Other RStudio settings (as described above) are saved.
- The R session is terminated.

2. Types of Data Objects in R

Before using R for the Data Analysis, you should know some key points to avoid getting many error messages. Writing variable names and identifying data types are two important points before starting the data analysis.

2.1 Writing variable names

- Since R is a Case Sensitive Language, variables 'Age' and 'age' will be treated as different variables in R. Therefore, you should select a common form to name all your variables. For example, use all simple for the variable names or the first letter capital and then simple for all others. Avoid using longer names.
  Example: To specify the variable height you can either use the variable name as height or Height
- If you want to add a variable name with two or more words insert `.` or `_` sign in between these words instead of a space.
  Example: To write the male height as a variable name you can use male.height, male_height or MaleHeight
- Do not include symbols such as `?, $,%, ^, &, *, (, ),-,#, ?,,,<,>, /, |, \, [ ,] ,{, and }` to variable names.

2.2 Check your data and help function in the relevant R package

- If there are any missing values in your data set, indicate them as `NA`.
- If you use any specific R packages for the data analysis, check the examples given in the help file of that package. For example, if you want to use `MASS package`, run the code `help(package="MASS")`.
- If you want to understand lm (linear model) function in the `MASS package` run the code `help(lm, package="MASS")`.
- In many help pages, there is an example which illustrates how the functions work. For example, if you want to execute examples relevant to **lm (linear model) function** run the command `??lm`.
- One of the most important points is to understand the data format required to use with the relevant package. Note that the functions given in the relevant package work only with this data format. Therefore, if your data set in not in that format, you have to **reshape** your data according to the required format.
- A dataset can be written in two different formats: **wide** and **long**. A **wide** format contains values that *do not* repeat in the first column. A **long** format contains values that *do* repeat in the first column.
- You may also need a particular **data structure** to use the relevant package.
- If you want to create a new variable, recode or rename variables.
  Refer:**Quick R**
- If you want to sort, subset or merge your data refer the following links: **sort**, **subset** **merge.**

Example for **wide** and **long** formats by consider the following two datasets that contain the exact same data expressed in different formats:

Wide Format

Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

2.3 Data Types

R has a wide variety of data types including scalars, vectors (numerical, character, logical), matrices, data frames, and lists.

Scalars

The simplest type of object is a scalar. A scalar is an object with one value. To create a scalar data object, simply assign a value to a variable using the assignment operator "<-".

Vectors

Vectors are the data objects probably most used in R. A vector can be defined as a set of scalars arranged in a one-dimensional array. Essentially a scalar is a one-dimensional vector. Data values in a vector are all the same mode, but a vector can hold data of any mode.

c()- function concatenates values

```
> temp <- c(3,2,4.3,-2,0) # numeric vector
> cities <- c("Kandy","Colombo","Matara","Galle","Gamapaha") # character vector
> status <- c(TRUE,TRUE,FALSE,TRUE,FALSE) #logical vector
```

Refer to elements of a vector using subscripts.

```
> temp [c(2,4)] # 2nd and 4th elements of vector
[1]  2 -2
> cities[1] #1st element
[1] "Kandy"
> length(status) #length of the vector
[1] 5
```

Matrices and Arrays

Matrices are collections of data values in two dimensions. An array is a matrix with more than two dimensions. Formatting data as matrices and arrays provides an efficient data structure to perform calculations in a computationally fast and efficient manner.

A Matrix is created using the **matrix()** function. The basic form of this function is `matrix(data, nrow, ncol, byrow, dimnames)`

```
# Create two 2x3 matrices.
> m<- matrix(c(3, 9, -1, 4, 2, 6), nrow = 2, ncol=3)
> print(m)
```

Data frames

Data frames are used to store tabular data in which each column contains values of one variable and each row contains a set of values from each column.

A data frame is created using the **data.frame()** function. In a data frame, the row names should be unique, and the data can be of numeric, factor or character type. Also, the number of observations of each column should be the same.

```
# Create a data frame having employer information
> emp.data <- data.frame( emp_id = c (1:5),
  emp_name = c("Saman","Nimal","Kumari","Sitha","Gerry"),
  salary = c(623.3,515.2,611.0,729.0,843.25))
> print (emp.data)
# Get the structure of the data frame.
> str(emp.data)
```

Lists

Lists contain elements of different types. For example, a list can contain numbers, strings, vectors and another list inside it. A list is created using **list()** function.

```
# Create a list containing a vector, a matrix and a list.
> list1 <- list(c("Jun","July","August"), matrix(c(1,3,9,-5,1,7), nrow = 2), list("red",75))
> print(list1)
```

- Inf which represents infinity; e.g. 1 / 0; Inf can be used in ordinary calculations;
- NaN represents an undefined value (\Not a Number"); e.g. 0 / 0; NaN
- NA a missing value

The # character indicates a comment. Anything to the right of the # (including the # itself) is ignored.

Factors

A factor is a special type of character vector. In most cases, character data is used to describe the other data, and is not used in calculations. However, for some computations, qualitative variables are used. To store character data as qualitative variables, a factor data type is used.

You may create a factor by first creating a character vector, and then converting it to a factor type using the factor () function:

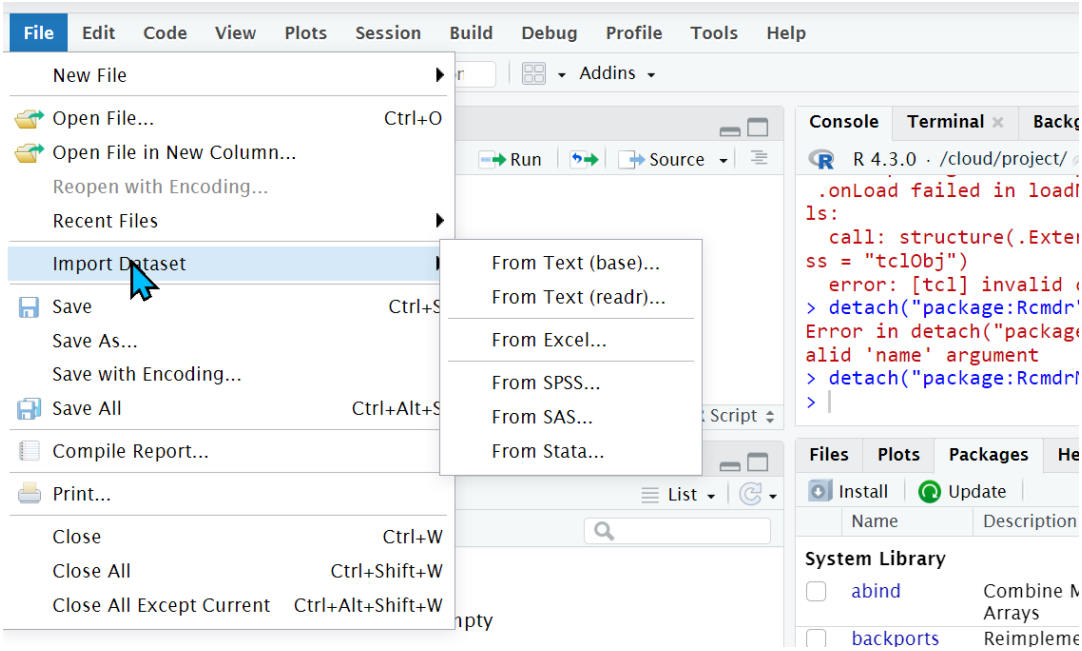
```
> settings<-c("High","Medium","Low")
> settings
[1] "High"   "Medium" "Low"
> settings<-factor(settings)
> settings
[1] High   Medium Low
Levels: High Low Medium
```

Notice that this creates "levels" based on the factor values (these are the values of categorical variables).

2.4 Importing data

The easiest way to enter data in R is to work with a text file, in which the columns are separated by tabs. There are many ways in which you can get such files. When importing data, always make sure that you never use column headings with a blank space in them, as R tends to interpret these as two different headings.

To import a data set, click File → Import Dataset, and then select the text or excel file that you want to import.



Example 1 (Ground Water Data)

Download the text file to your current working directory which contains the data with the name "GroundWater.txt" from Google Drive using the below link: https://drive.google.com/file/d/1KBHNKIGEPYsJMzTeZChlDM7VxT-_YtA1/view?usp=drive_link

The given data set includes area (A, B, C, D), pH levels and concentrations of naturally occurring arsenic (units are parts per billion) in groundwater for a random sample of 102 wells in the given Province of Sri Lanka. A pH less than 7 is called acidic, and a pH above 7 is called alkaline. Too high a pH means the water is unusable or needs expensive treatment to make it usable. Also, a mean arsenic level of less than 8.0 parts per billion (ppb) is considered safe for use.

- Import the data into R workspace.
- Check your data set using the below R codes.

```
> library("readr")   # Install readr package
>  GroundWater <- read.delim(file.choose())

> dim(GroundWater) # Retrieve or set the dimension of an object.
> nrow(GroundWater) # Returns the number of rows in a Data Frame
> ncol(GroundWater) # Returns the number columns in a Data Frame
> head(GroundWater) # Displays the first 6 rows of a Data Frame
> tail(GroundWater) # Displays the last 6 rows of a Data Frame
> colnames(GroundWater) # Retrieve column names of a Data Frame
```

2.5 Exporting data

1. To write data to a txt file having tab separated values, use `write.table(data, file = "data.txt", sep = "\t", row.names = TRUE, col.names = NA)`.
2. To write data to a comma delimited (csv) file, use `write.csv(data, file = "data.csv")`.
3. To write data to a new xlsx workbook, use
   `library("xlsx")`
   `write.xlsx(data1, file = "dataworkbook1.xlsx",sheetName = "firstsheet", append = FALSE)`.
   Then add a second worksheet to the same workbook using `write.xlsx(data2, file = "dataworkbook2.xlsx",sheetName="secondsheet", append=TRUE)`

3. R Packages

3.1 What is a package?

Packages are basically collections of functions. R comes with some basic packages needed for data analysis. You can download and install additional packages to run specific analyses or make specific graphs.

Here are some examples of packages with details:

1. *ade4*: Analysis of Ecological Data
2. *seqinr*: Biological Sequences Retrieval and Analysis
3. *qcc:* Quality Control Charts
4. *geoR*: Analysis of geostatistical data
5. *vegan*: Community Ecology Package
6. *Rcmdr*: R Commander is a GUI (Graphical User Interface) for the R programming language based on the *tcltk* package.

Currently, the CRAN package repository features, **19597** available packages. To see all the available CRAN packages by name visit:

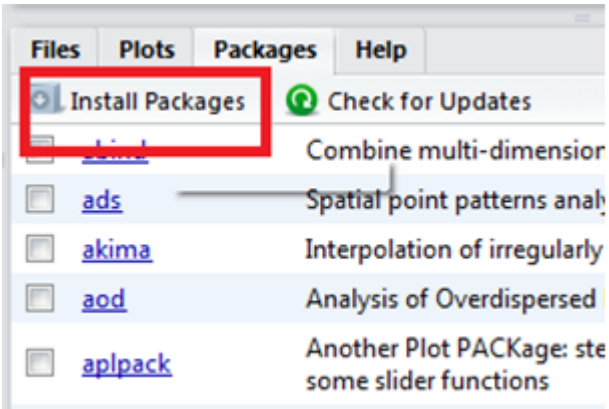
http://cran.r-project.org/web/packages/available_packages_by_name.html

3.2 Installing packages in RStudio.

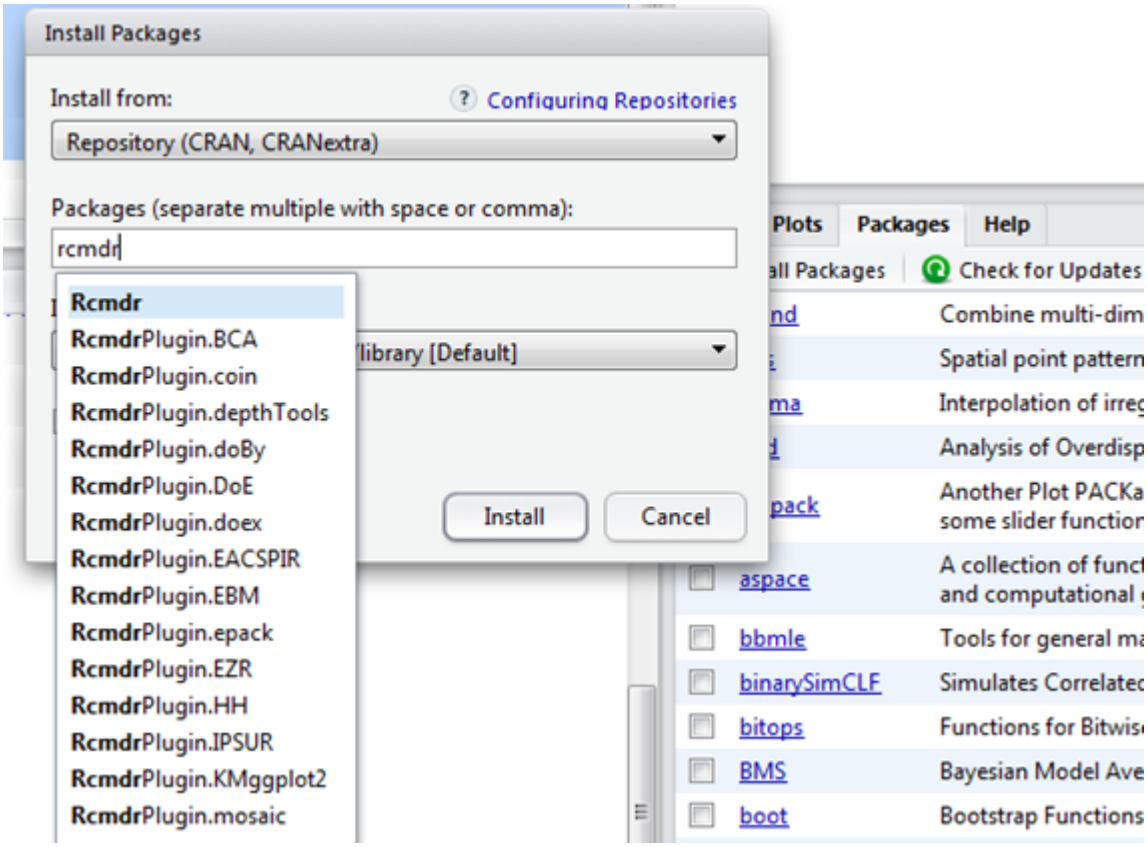
There are two methods to install R packages in Rstudio.

Method 1: Install from CRAN(Web) directly

- Go to the "Packages" tab in **Bottom right** pane of RStudio**,** and click on "Install Packages".
- The first time when you do this, you'll be prompted to choose a CRAN mirror. R will download all necessary files from the server you select here. Choose the location closest to you.



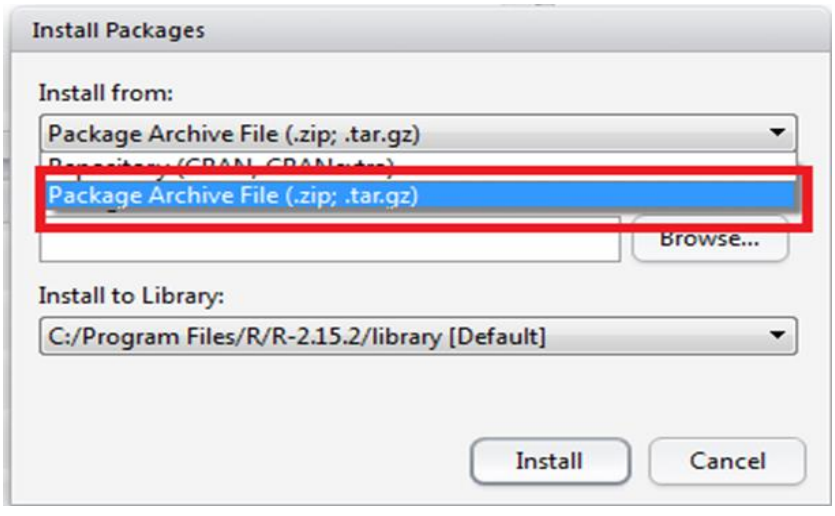
- Start typing the name of the packages (for example: “Rcmdr”) until you see it appear in a list. Select the required package, ensure that “Install dependencies” is checked, and click “Install”.



- Wait while all the dependent parts of the package are installed.

Method 2: Install from the source

- Download the add-on R package’s zip file from CRAN (www.cran.r-project.org/web/packages), say mypkg.zip, and store it in your working directory.
- Go to “Packages” “Install Packages” “Packages Archive File”

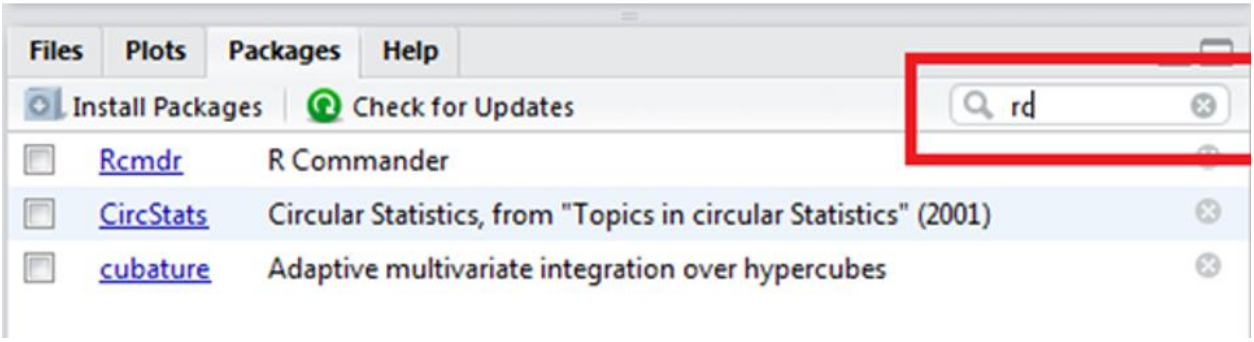


- Browse the saved zip file and click on install button.

3.3 Loading Packages

Whenever you want to use an R package you must not only have installed it locally, you must also load the package during the session you are using it. This makes R more efficient and uses less overhead than if all installed packages are loaded every time you use R, but makes the user do a little more work. After the required packages are installed, you need to load the package in order to work with it. To load a particular package in RStudio:

- Go to the "Packages" tab and type the name of the Package in the search bar.



- Tick the check box near by the name of the package in the resulting list. Then the package will be loaded.
- You can also do this by adding the code ***library(name of the package)*** to your R script file.

4. R Markdown, R Notebook and Quarto

4.1 What is R Markdown?

R Markdown is a markup language and framework that allows to create dynamic, reproducible documents. It combines the simplicity and readability of plain text with the ability to embed code, generate visualizations, and produce high-quality reports in various output formats such as HTML, PDF, and Word.

R Markdown is an extension of Markdown markup language used for formatting text. Markdown provides a simple syntax for creating headers, lists, links, and other text formatting elements. R Markdown enhances this functionality by enabling the inclusion of R code chunks within the document.

With R Markdown, you can seamlessly interweave narrative text, code, and output. This makes it an excellent tool for reproducible research, data analysis, and report generation. By embedding executable code within the document, R Markdown enables the automatic execution of code chunks, ensuring that the results and visualizations are always up to date.

One of the key advantages of R Markdown is its flexibility. It supports a wide range of programming languages, including R, Python, Julia, and SQL, allowing you to leverage the power of different languages within a single document. Additionally, R Markdown supports the integration of various data visualization libraries, such as ggplot2 and Plotly, enabling the creation of interactive and dynamic visualizations.

R Markdown documents are authored using plain text files with a `.Rmd` extension. These files can be easily version-controlled, shared, and collaborated on using tools like Git and GitHub. The R Markdown file is then processed by the knitr package in R, which executes the code chunks, captures the output, and generates the final document in the desired output format.

Overall, R Markdown provides a powerful and flexible framework for creating reproducible reports, data analyses, and presentations. It promotes a seamless workflow between code, data, and narrative, making it an invaluable tool for researchers, data scientists, and anyone looking to communicate their findings effectively.

To learn some R Markdown basics, refer https://pushpa-wijekoon.netlify.app/post/learn-r/learnr/.

4.2 What is R Notebook?

R Notebook is an interactive computational environment within the RStudio IDE that combines the functionality of R code execution, Markdown formatting, and inline output display. It provides a user-friendly interface for data analysis, exploration, and documentation, allowing users to create dynamic and reproducible documents.

Similar to R Markdown, R Notebooks enable the integration of narrative text, code chunks, and output. However, the key distinction is the interactive nature of R Notebooks. Users can execute code chunks in a step-by-step manner, observing the immediate results and modifying the code as needed. This interactive feature makes R Notebooks particularly useful for exploratory data analysis and iterative workflows.

When creating an R Notebook, you start with a plain text file with a `.Rmd` extension, similar to R Markdown. However, within the notebook, you can divide your content into different chunks using code and Markdown cells. Code chunks are denoted by three backticks followed by an "r" within curly braces, and Markdown cells are enclosed in three backticks without any language specification.

As you work through the notebook, you can execute individual code chunks or run all the code in a specific section. The results of the code execution, including tables, plots, and other output, are displayed inline within the notebook. This allows for immediate visualization and exploration of the data.

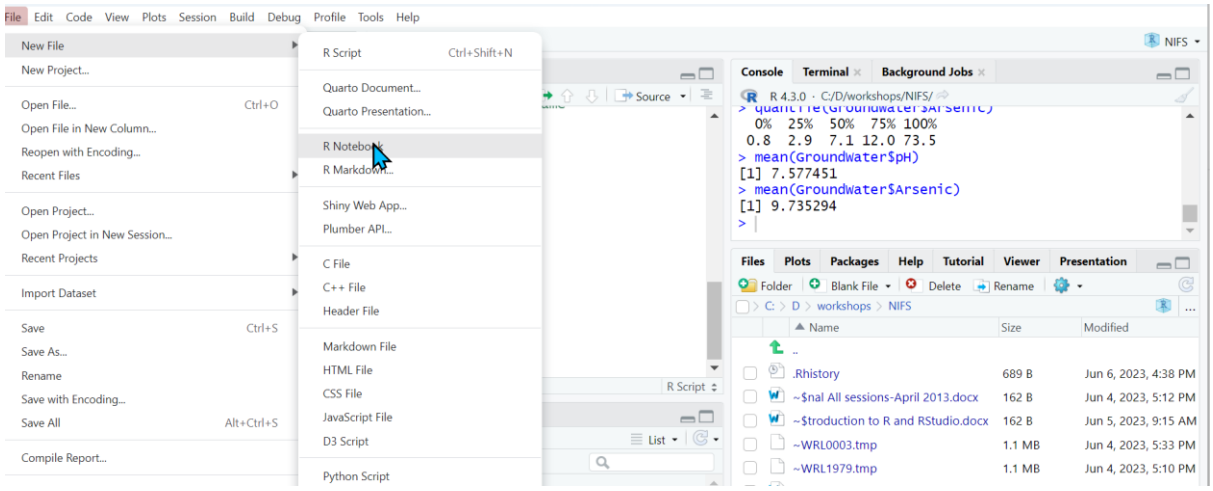
R Notebooks also provide a convenient way to mix and match R code with other languages such as Python or SQL. By using the appropriate language specification within code chunks, you can seamlessly execute code written in different languages within the same document.

Another valuable feature of R Notebooks is the ability to create interactive visualizations using packages like plotly and ggplot2. These interactive plots allow users to explore the data further by zooming, panning, and hovering over elements for additional information.

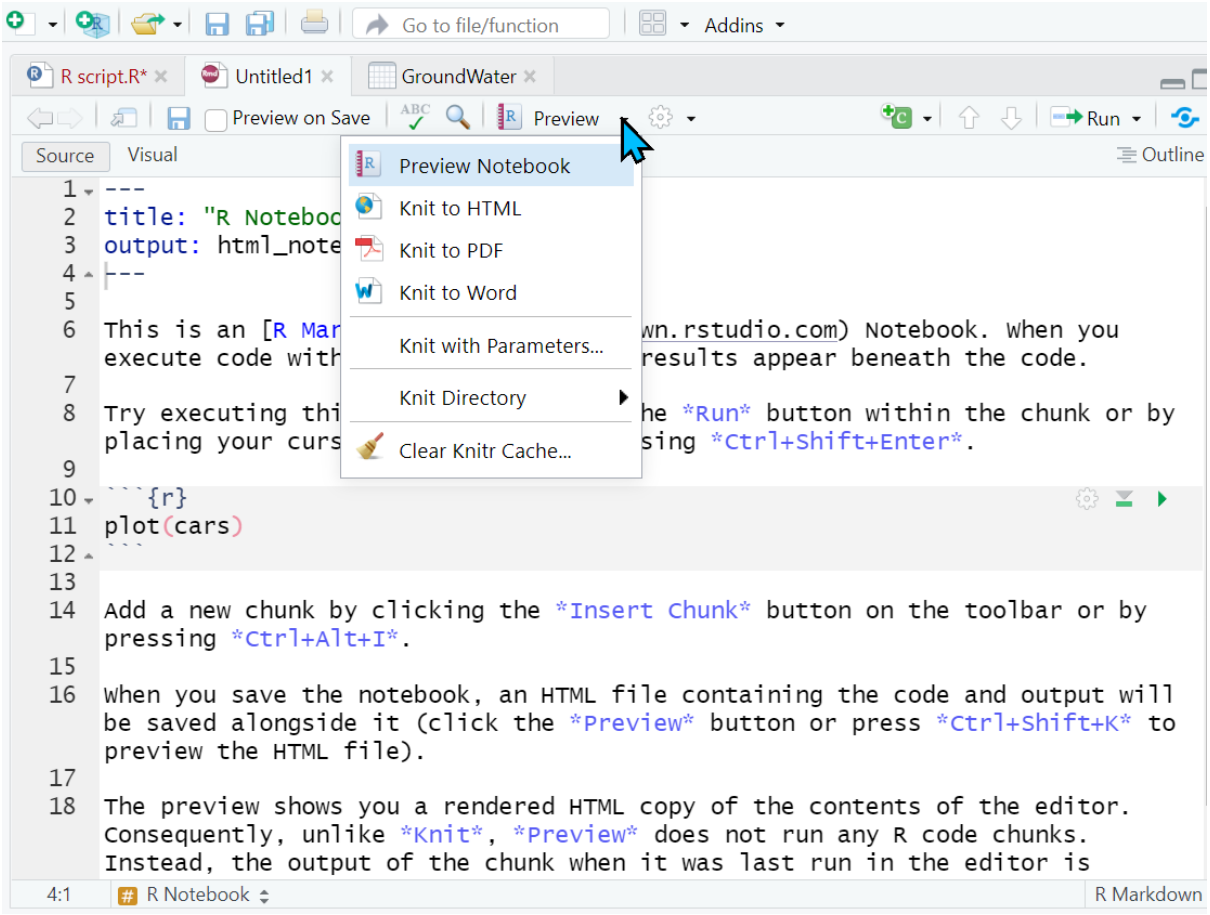
Once you have completed your analysis and documentation, you can easily share your R Notebook with others. The notebook can be saved as an `.Rmd` file or exported to different formats such as HTML or PDF. This enables reproducibility and facilitates collaboration among researchers and data scientists.

In summary, R Notebook provides an interactive and flexible environment for data analysis, exploration, and documentation. By combining code execution, Markdown formatting, and inline output display, it allows users to iteratively work with data, visualize results, and communicate findings effectively.

To open R Notebook select **File → New File → R Notebook**



Then, you have the default R Notebook with some R codes as below:



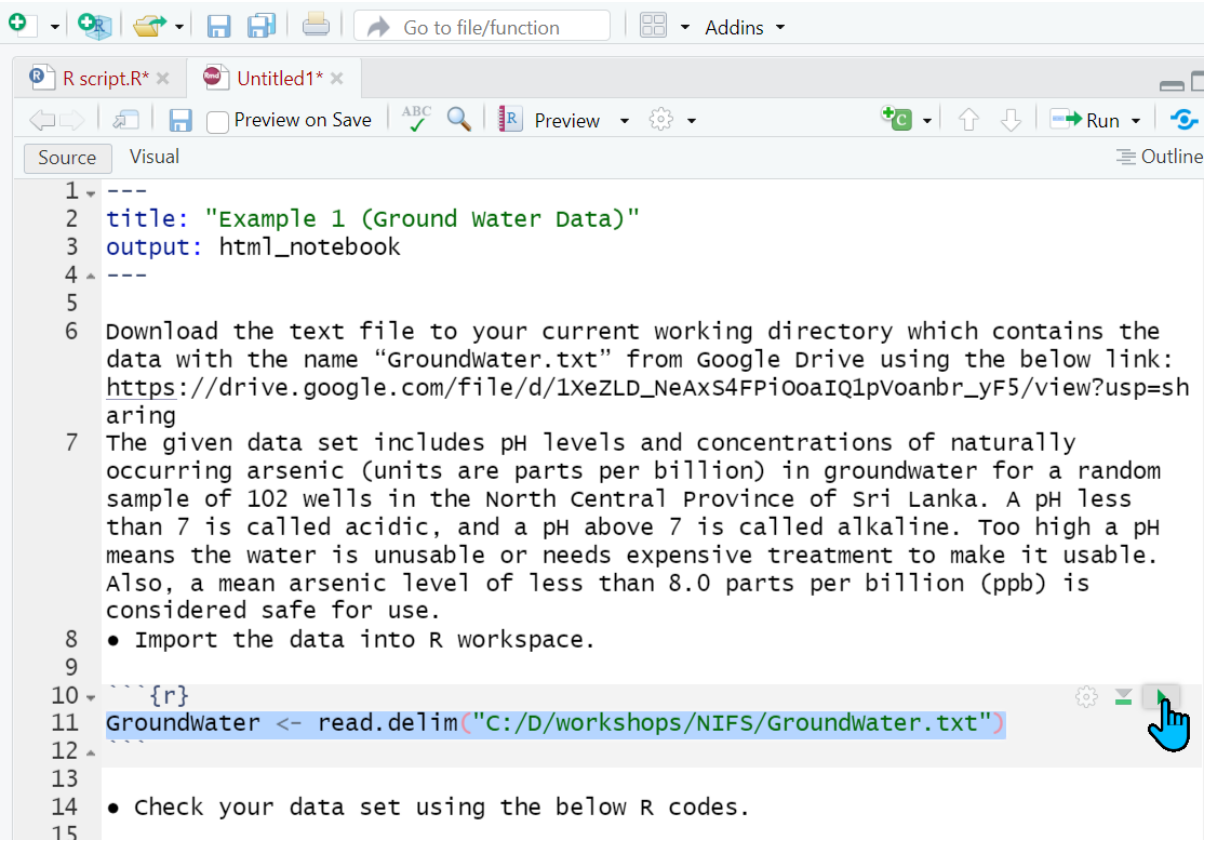
When you click “Preview”, you can see several options.

Selecting “Preview Notebook” you can save the file and view the output as an HTML file. This can also be done by selecting “Knit to HTML”. Further, by clicking “Knit to PDF” or “Knit to Word” you can create a pdf or a word document with your output.

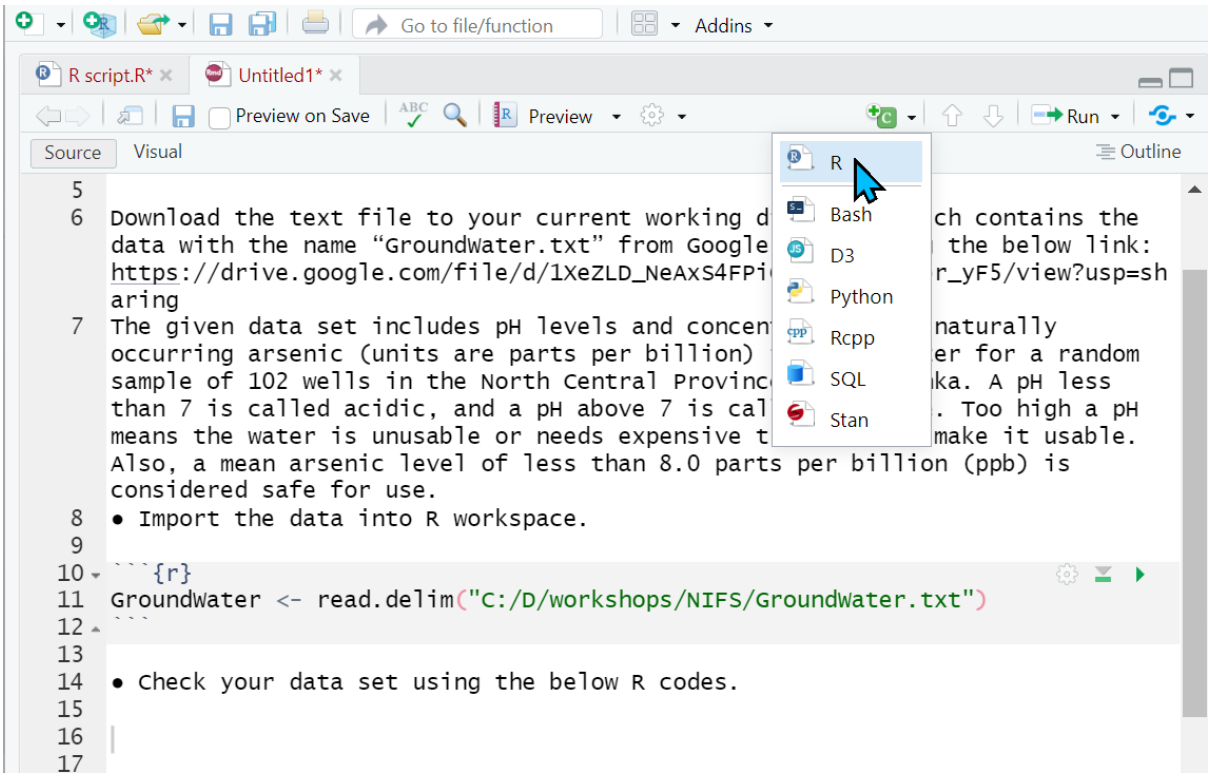
Note that in the output file you can see the text, R codes and other outputs as well. Therefore, using R Notebook, you can easily create a report to share with others when doing the data analysis.

Example 2

- Create R Notebook to show the texts and R codes in Example 1.
- Give the title for the R Notebook as **Example 1 (Ground Water Data)**.
- Add the description of the data set at the top as given in Example 1.
- Import the data set typing the below R code within the block ```{r} ```
  GroundWater <- read.delim("C:/D/workshops/NIFS/GroundWater.txt")
- To run the R code, click the green arrow as shown in the below figure in the R code chunk.



- To add more R code chunk, click the green button as shown in the below figure and select R.



- Give interpretations of the output that you obtained as a text.
- Create a Notebook in Word.

4.3 What is Quarto documents?

Quarto documents enable users to create dynamic reports, presentations, and websites by integrating codes and results together. Quarto supports multiple programming languages, such as R and Python.

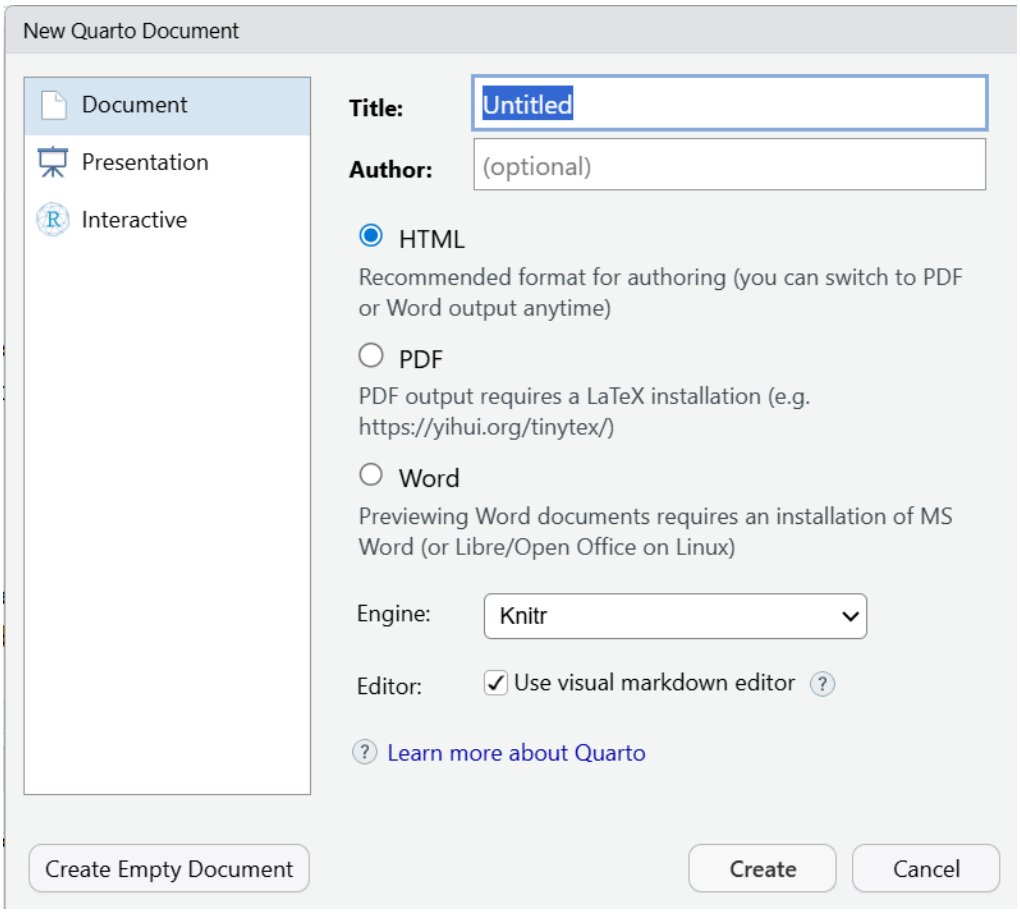
Quarto documents use a .qmd (Quarto Markdown) extension, a plain text format that blends markdown text with executable code chunks.

Code blocks are executed during the rendering process, and their results (e.g., plots, tables, text) are automatically embedded in the final document.

More details: https://quarto.org/

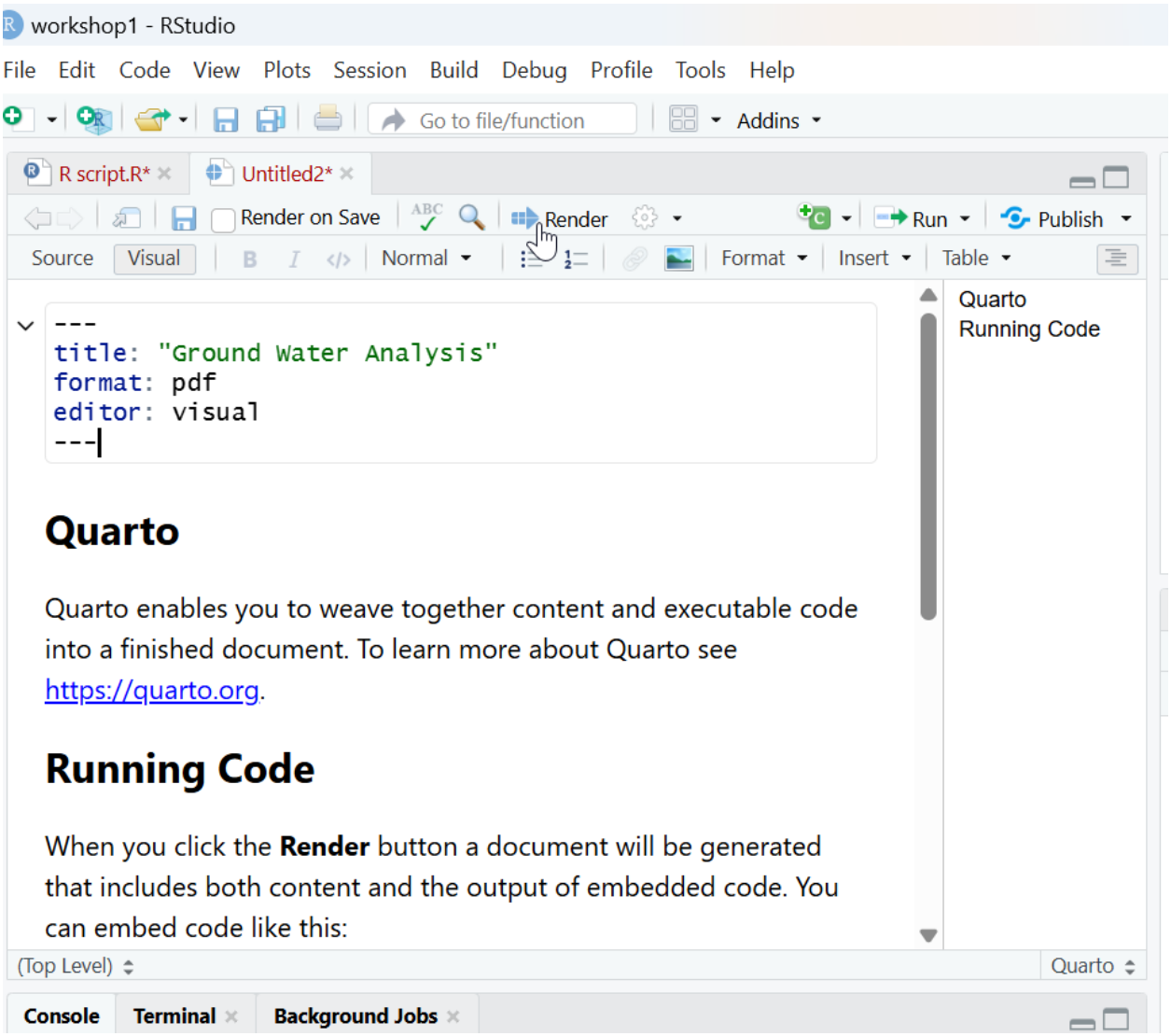
To open a Quarto document select

File → New File → Quarto Document



- Add the Title
- Select the document type and create.

Then, you have the default Quarto document with some texts and R codes as below:



- Just like in Word, you can type text here
- Also, in R code chunks, you can add your R codes.
- To create and save the document, click **Render** button

Example 3

- Create a Quarto word document to show the texts and R codes in Example 1.
- Give the title as **Example 1 (Ground Water Data)**.
- Add the description of the data set at the top as given in Example 1.
- Import the data set typing the below R code within the block ```{r} ```
  GroundWater <- read.delim("C:/D/workshops/NIFS/GroundWater.txt")
- To run the R code, in R chunk, click the green arrow, and click Render button to create the document.

5. Descriptive Summary Statistics

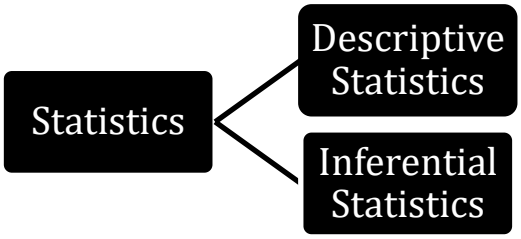
What is Statistics?

Statistics is a science that deals with collection, organization, summarization, analysis, presentation and making inference on data.

Why Statistics?

In almost any area of work – business, medicine, engineering, education, social sciences, and physical sciences etc. – you must be able to read, interpret, and apply the results of a statistical analysis of research data.

5.1 Two Branches of Statistics:



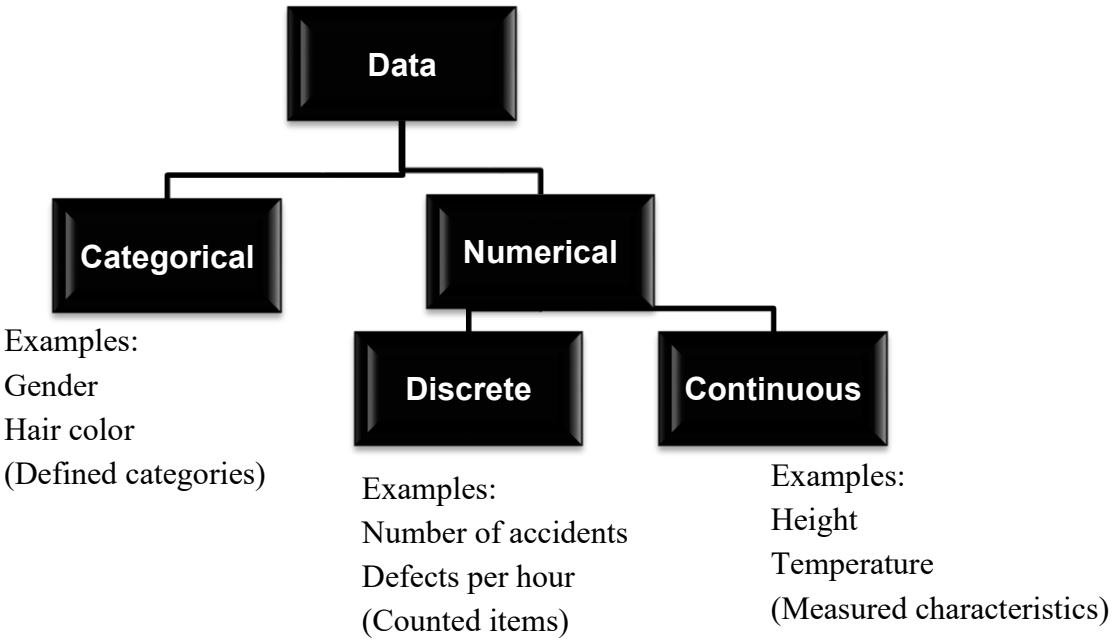
- **Descriptive Statistics**
  Consist of collection, organization, summarization and presentation of data.
- **Inferential Statistics**
  Consists of generalizing from samples to populations, estimating parameters, performing hypothesis testing, determining relationships among variables, and predictions.

Terminology

- A statistical problem involves studying some characteristics associated with a group of objects commonly called **experimental units** or **subjects**.
- A characteristic being studied that is associated with each experimental unit is called a **variable**.
- The collection of values measured on experimental units (called observations) associated with the variable in the study is called a **data set**.
- **Population:** Collection of all the objects or items that are of interest in a statistical study. The individual objects in the population are the experimental units or subjects
- **Sample:** A finite portion (subset) of the population that is used to study characteristics of concern in the population
- **Parameter:** Characteristic or measure obtained from a population.
- **Statistic** (not to be confused with Statistics): Characteristic or measure obtained from a sample.
- **Census:** The collection of data from every element in the population.
- **Sampling:** The collection of data from the elements of a sample.
- The number of times a specific category (or value) is observed is called **frequency**.

5.2 Types of Data

The main categories of data types are shown below:



Univariate, Bivariate and Multivariate cases

- If you only measure one variable per sampling unit, the data set is univariate.
- Data set having two variables per unit is bivariate.
- While data for three or more variables measured on the same sampling unit are multivariate.

5.3 Descriptive Statistics

Descriptive statistics involves methods used to summarize and describe the main features of a dataset providing a concise and meaningful understanding of the characteristics of data. Statisticians commonly try to describe the collected data with

- Measures of Central Tendency (measures of averages): Mean, Median, Mode etc
- Measures of Relative Position (Relative position of a specific value in comparison with other data): Quartiles, deciles or percentile
- Measures of Dispersion (Spread of data): Range, variance, standard deviation
- Measures of Shape (shape of the distribution): Skewness, Kurtosis

The main objective of statistics is to draw generalizations about a population based on the data collected from a sample. In most situations it is difficult to work with the complete distribution of values. Instead, distributions can be represented using summery measures.

These summary measures will be different for different samples. Also, they will identify differences of different distributions leading to an analysis of data.

5.3.1 Measures of Central Tendencies

The central tendency of a distribution locates the "center" of a distribution of values.

Mean

This is also known as the arithmetic average, and it is sensitive to outliers.

$$Sample\ Mean(\bar{X}) = \frac{X_1 + \cdots + X_n}{n} = \frac{\sum_{i=1}^{n} X_i}{n}$$

n = total number of observations in the sample

$$Population\ mean\ \mu = \frac{\sum_{i=1}^{N} X_i}{N}$$

N= total number f observations in the population

Median

- The median is the central value of the set of data when the values are arranged in ascending or descending order of magnitude.
- The median divides the distribution into two equal parts (divides the bottom 50% of the data from the top 50% of the data.). The median is, thus, a potential average

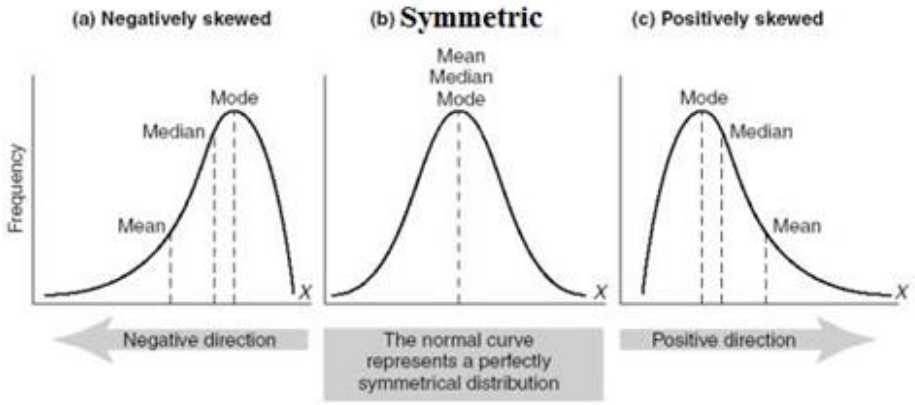
To compute the median, it is necessary that the observations to be arranged in ascending or descending order. The median is a robust alternative to mean.

Mode

Within a set of value the value with the highest frequency is defined as Mode.

There can be one or more modes or no modes in a given set of data

- Distributions with only one mode is known as unimodal distribution
- Distributions with only two modes is known as bimodal distribution
- Distributions with more than two modes in known as multimodal distribution



- Symmetrical distribution $\Rightarrow$ Mode ≈ Median ≈ Mean
- When the data are negatively skewed: $\Rightarrow$mean < median < mode.
- When the data are positively skewed: $\Rightarrow$ mean > median > mode.

5.3.2 Measures of Relative Position

Quartiles

Quartiles are the points that divide an array of numbers into four equal parts. There are three points that divide a group of numbers into two equal parts

- first quartile (designated Q_1) = lower quartile = cuts off lowest $1/4^{th}$ of data

  $= value\ of\ \frac{n+1^{th}}{4}\ item$

- second quartile (designated Q_2) = median = cuts data set in half

  $= value\ of\ \frac{n+1^{th}}{2}\ item$

- third quartile (designated Q_3) = upper quartile = cuts off lowest $3/4^{th}$ of data

  $= value\ of\ \frac{3(n+1)^{th}}{4}\ item$

Deciles

Deciles divide the data set into 10 equal parts. $D_1 – D_9$

Percentiles

Percentiles divide the data set into 100 equal parts. $P_1 – P_{99}$.

The **minimum**(min) is the smallest value of the variable, and the **maximum**(max) is the largest value of the variable in a sample. The minimum and maximum along with quartiles (Q_1, Q_2, and Q_3) are known as **five-number summary.**

5.3.3 Measures of Variation

Measures of Variation or Spread includes the range, interquartile range (IQR) and standard deviation (variance). These numerical summaries describe the amount of spread that is found among the data, with larger values indicating more variability.

Range

Range = highest value – lowest value, a relatively crude estimator.

Interquartile Range

IQR = Q3 - Q1, measures the spread of the middle (usually most common) 50% of the data

Variance and Standard deviation

$$Population\ Variance\ (\sigma^2)\ =\ \frac{\sum_{i=1}^{N}(x_i-\mu)^2}{N}$$

where μ = Population mean and N= Population size

$$Sample\ Variance\ (S^2)\ =\ \frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n-1}$$

✓ Standard Deviation (SD) is the positive square root of the variance

- Higher values for both of these indicators indicate a larger amount of variability than do lower numbers.
- Zero stands for no variability at all (e.g., for the data 3, 3, 3, 3, 3, 3, the variance and standard deviation will equal zero).
- When no variability, the numbers are a constant (i.e., the same number).
- The variance tells you the average deviation from the mean, in "squared units."
- The standard deviation tells you how far the numbers tend to vary from the mean.

Coefficient of variation – CV

The coefficient of variation (CV), also known as "relative variability", equals the standard deviation divided by the mean

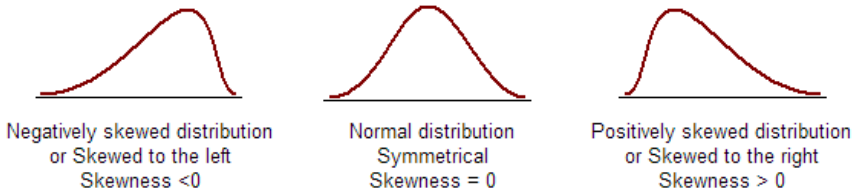
$$\text{Sample } CV = \frac{SD}{\bar{X}} \times 100\ \%$$

5.3.4 Measures of Shapes

Coefficient of Skewness

Coefficient of Skewness(S_k) is a measure of the asymmetry of the distribution. A distribution with a significant positive skewness has a long right tail. A distribution with a significant negative skewness has a long left tail. For a skewed distribution, the mean tends to lie on the same side of the mode as the longer tail.

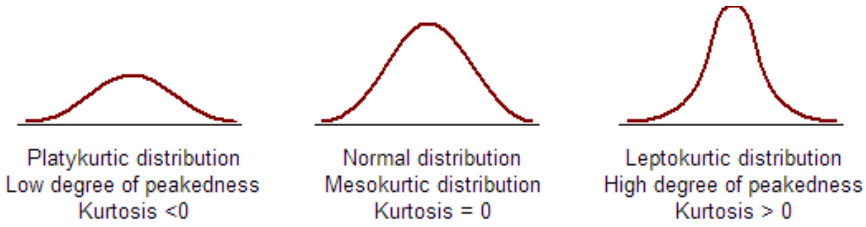
- $S_k = 0$ $\Rightarrow$ symmetrical distribution
- $S_k > 0$ $\Rightarrow$ positively skewed distribution
- $S_k < 0$ $\Rightarrow$ negatively skewed distribution



Coefficient of Kurtosis

Coefficient of Kurtosis (κ) is the degree of peakedness in a distribution, usually taken relative to a normal distribution. The peakedness property means that there is an excess frequency at the center of the distribution.

- $\kappa = 0$ $\Rightarrow$ Mesokurtic
- $\kappa > 0$ $\Rightarrow$ Leptokurtic
- $\kappa < 0$ $\Rightarrow$ Platokurtic

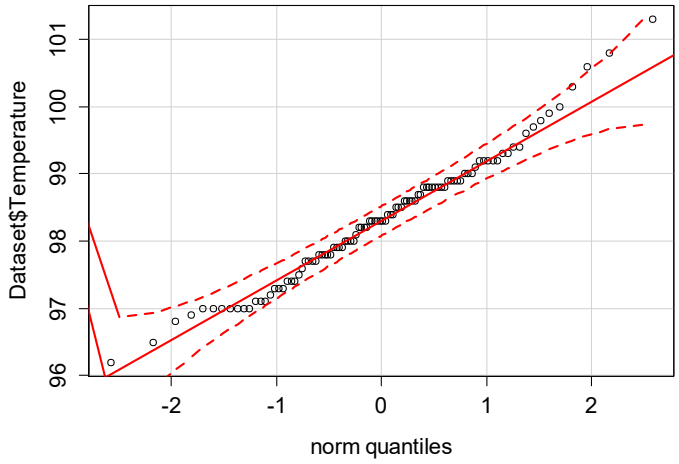


5.4 Normality tests

Normality tests are used to test whether the observed data are distributed in a way that is consistent with a normal distribution. The validity of this assumption is to be checked for most of the data analysis methods.

Graphical method – Normal Probability plot

The data are plotted against a theoretical normal distribution in such a way that the points should form an approximate straight line. Departures from this straight line indicate departures from normality.



In addition to the graphical method, several tests are available. **Shapiro–Wilk, Anderson-Darling and Lilliefors** tests is three of those tests. A small *p*-value ($p < 0.05$) is evidence that your data were sampled from a non-normal distribution. A large *p*-value means that your data are consistent with a Normal distribution.

- **Shapiro–Wilk Test** - `shapiro.test(GroundWater$pH)`
- Use the following R-codes from the "nortest" package for **Anderson-Darling and Lilliefors** tests.
  `ad.test(GroundWater$pH), lillie.test(GroundWater$pH)`

Example 4

Consider again the GroundWater data set. Compute the summary statistics and test the normality of the two variables using the below codes.

```
> summary(GroundWater)
> mean(GroundWater$pH)
> mean(GroundWater$Arsenic)
> range(GroundWater$pH)
> range(GroundWater$Arsenic)
> min(GroundWater$pH)
> max(GroundWater$Arsenic)
> quantile(GroundWater$pH)
> quantile(GroundWater$Arsenic)
> IQR(GroundWater$pH)
# Select numeric variables
> GW<-data.frame(GroundWater$pH, GroundWater$Arsenic)
```

```
> summary(GW)
> cov(GW)
> cor(GW)
# Testing the Normality
> shapiro.test(GroundWater$pH)
> library(nortest)
> ad.test(GroundWater$pH)
> lillie.test(GroundWater$pH)
> library("car")
> qqPlot(GroundWater$pH)
> qqPlot(GroundWater$ Arsenic)
```

Example 5 (Body Temperature Data)

An Excel file which contains the data with the name "BodyTemperature.xlsx" is stored on your current working directory. This file includes Gender, Age, Heart rate, and normal Body temperature of 100 adults between the age of 20 and 50.

(i) Open a new Quarto document and import this excel data into R workspace.

(ii) For each numerical variable, obtain the minimum, maximum, quartiles, IQR, mean, variance and the number of missing values.

(iii) For each categorical variable (defined as a factor), reports the number of observations at each level of the factor.

(iv) Find the coefficient of variation for Age and Temperature variables by gender and comment.

(v) Find the frequency counts of the Gender variable.

6. Data Visualization

Graphs are organized by

- the number of variables to be plotted.
- the type of variables to be plotted.
- the purpose of the visualization.

6.1 Univariate graphs

Univariate graphs plot the distribution of data from a single variable. The variable can be categorical (e.g., race, sex) or quantitative (e.g., age, weight).

6.1.1 Univariate graphs for categorical data

The distribution of a single categorical variable is typically plotted with a bar chart, a pie chart, or a tree map.

You can draw bar charts for counts or percentages, and each bar represents a categorical variable. Pie charts are controversial in statistics. If your goal is to compare the frequency of categories, use a bar chart. If your goal is to compare each category with the whole, and the number of categories is small, then use pie charts.

An alternative to a pie chart is a tree map. Unlike pie charts, it can handle categorical variables that have *many* levels.

Example 4: Marriage Dataset

The `Marriage dataset` is a dataframe comes from the mosaicData package. It contains the marriage records of 98 individuals collected in Alabama for the following 15 variables.

- `bookpageID` a factor with levels for each book and page (unique identifier)
- `appdate` date on which the application was filed
- `ceremonydate` date of the ceremony
- `delay` number of days between the application and the ceremony
- `officialTitle` a factor with levels `BISHOP`, `CATHOLIC PRIEST`, `CHIEF CLERK`, `CIRCUIT JUDGE`, `ELDER`, `MARRIAGE OFFICIAL`, `MINISTER`, `PASTOR`, `REVEREND`
- `person` a factor with levels `Bride`, `Groom`
- `dob` a factor with levels corresponding to the date of birth of the person
- `age` age of the person (in years)
- `race` a factor with levels `American Indian`, `Black`, `Hispanic`, `White`
- `prevcount` the number of previous marriages of the person, as listed on the application
- `prevconc` the way the last marriage ended, as listed on the application
- `hs` the number of years of high school education, as listed on the application
- `college` the number of years College education, as listed on the application. Where no number was listed, this field was left blank, unless less than 12 years High School was reported, in which case it was entered as 0.
- `dayOfBirth` the day of birth, as a number from 1 to 365 counting from January 1

- `sign` the astrological sign, with
  levels `Aquarius` `Aries` `Cancer` `Capricorn` `Gemini` `Leo` `Libra` `Pisces` `Sagittarius` `Scorpio` `Taurus` `Virgo`

Note that `officialTitle` is a categorical variable with 7 levels.
Install the mosaicData R package. Open a new R Notebook and access the dataset using below codes.

```
library(dplyr)
  data(Marriage, package="mosaicData")
  head(Marriage)
glimpse(Marriage)
summary(Marriage)
```

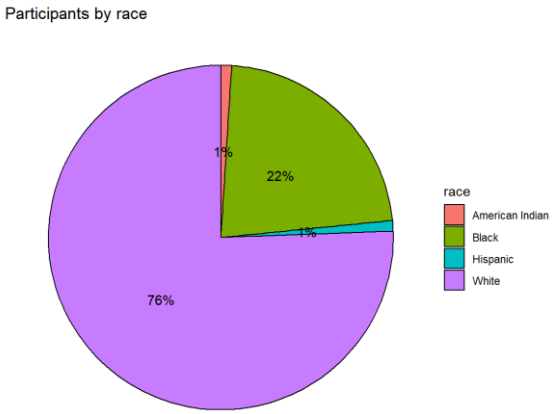
Look at the variable types; Here we have
(i) fct: factor variable type
(ii) date: date variable type
(iii) int: integer variable type
(iv) dbl: double class variable type, which is a data type defined to hold numeric values with decimal points.

Drawing a pie chart

Use pie charts if you want to compare each category with the whole, and the number of categories is small.

To draw a pie chart to show the percentage of race variable which includes only four categories (American Indian, Black, Hispanic, White), use the below R codes:

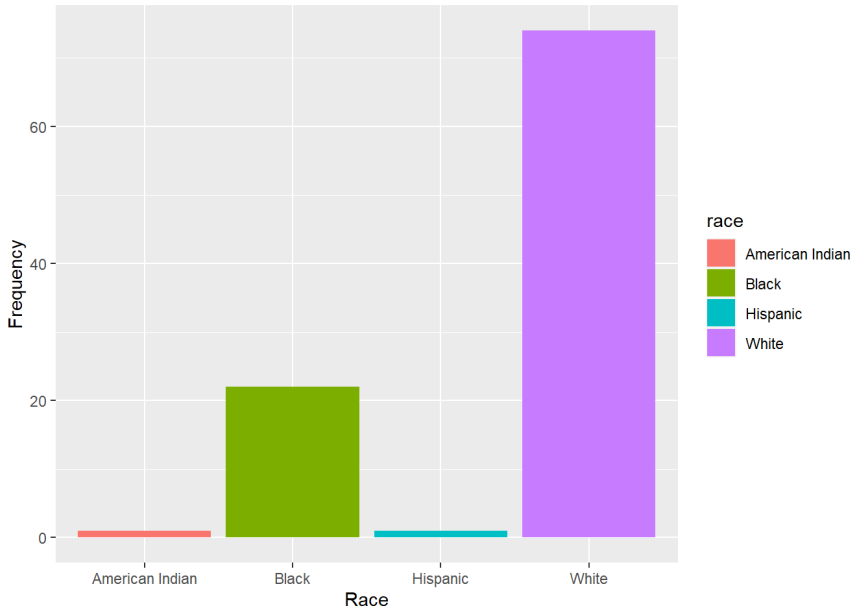
```
# create a pie chart of marriage officials
library(ggplot2) # Install ggplot2 package
plotdata <- Marriage %>%
  count(race) %>%
  arrange(desc(race)) %>%
  mutate(prop=round(n*100/sum(n),1), lab.ypos=cumsum(prop)-0.5*prop)
plotdata$lable<- paste0(round(plotdata$prop), "%")
ggplot(plotdata,
       aes(x="", y=prop, fill = race)) +
  geom_bar(width=1, stat="identity", color="black") +
  geom_text(aes(y=lab.ypos, lable=lable1), color="black")+
  coord_polar("y", start=0, direction=-1)+
  theme_void()+
  labs(title = "Participants by race")
```



Drawing a Frequency Bar chart

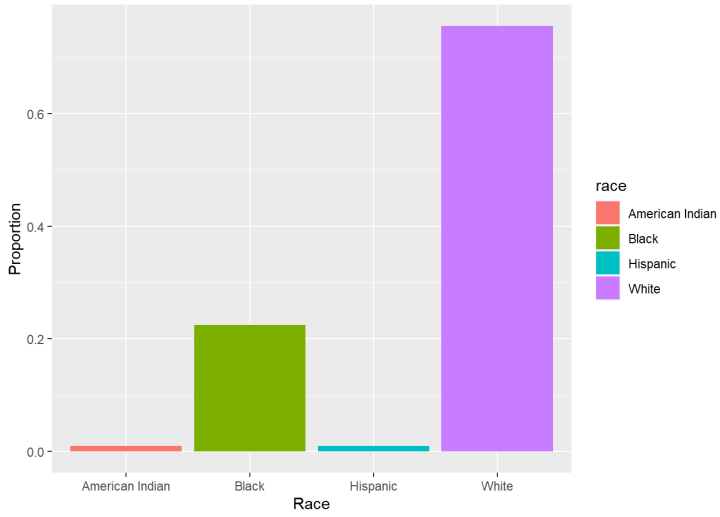
Bar graphs are crucial for visually presenting and making sense of categorical data. They give a concise and clear summary of how the categories are distributed, enabling quick comparisons and the discovery of patterns or trends.

```
# Frequency chart
Marriage%>%
  ggplot(aes(x = race, fill = race)) +
  geom_bar() +
  labs(y = "Frequency", x = "Race")
```



Drawing a Proportion Bar chart

```
Marriage %>%
  ggplot(aes(x = race, y = ..count../sum(..count..),fill = race)) +
  geom_bar() +
  labs(y = "Proportion", x= "Race")
```

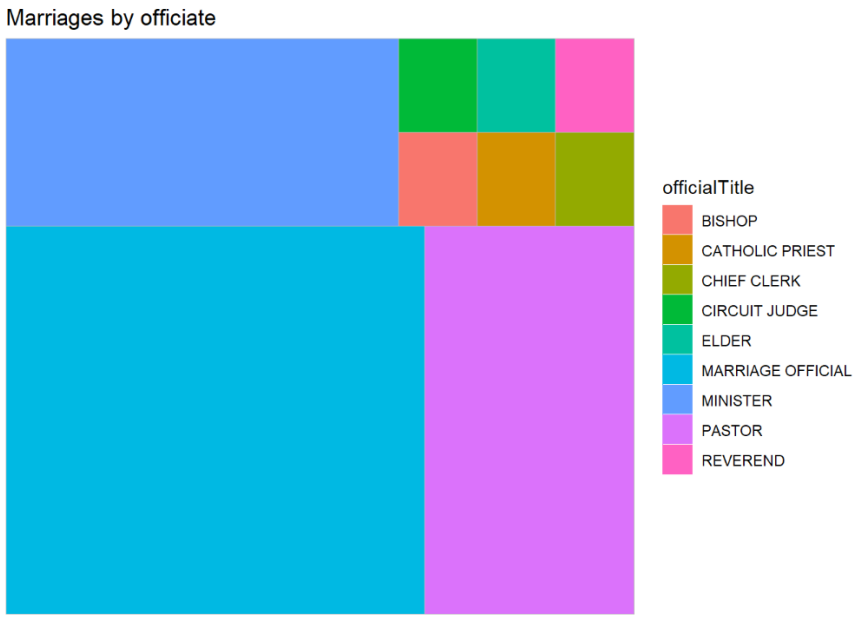


Drawing a Tree map

An alternative to a pie chart is a tree map. Unlike pie charts, it can handle categorical variables that have many levels. The following tree map shows the marriage counts for the variable officialTitle.

```
# create a treemap of marriage officials
library(treemapify)
plotdata <- Marriage %>%
  count(officialTitle)

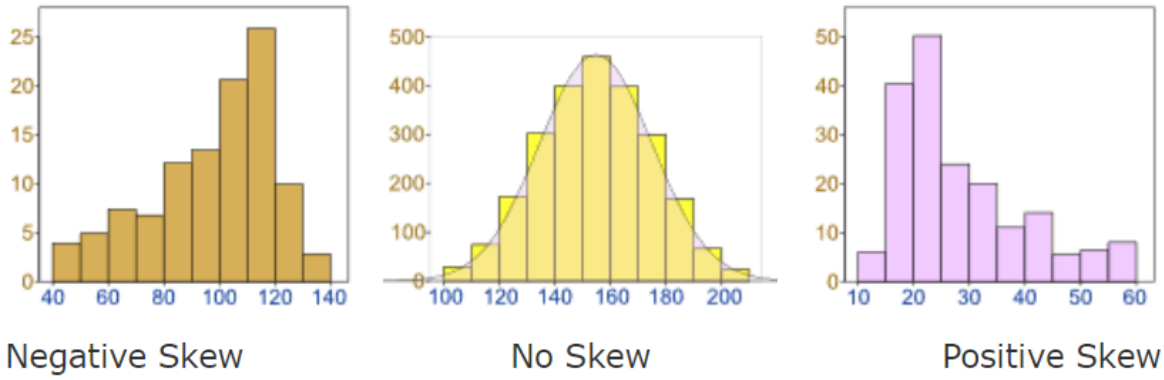
ggplot(plotdata,
       aes(fill = officialTitle,
           area = n)) +
  geom_treemap() +
  labs(title = "Marriages by officiate")
```



6.1.2 Univariate graphs for quantitative data

The distribution of a univariate quantitative variable can be plotted with a histogram, box plot, kernel density plot, or dot plot.

The shape of the data set can be identified by drawing a histogram. It is helpful to try smaller and larger number of bins for a histogram to get a better impression of the shape of the distribution.



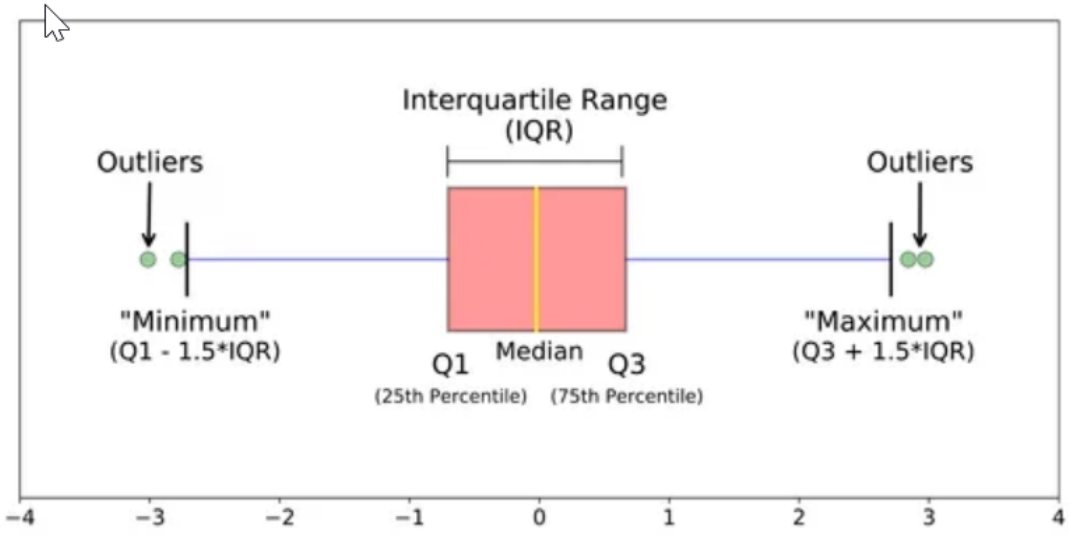
Drawing a histogram

Draw a histogram for the variable `age` (age of the person (in years) in `Marriage` dataset, and interpret results.

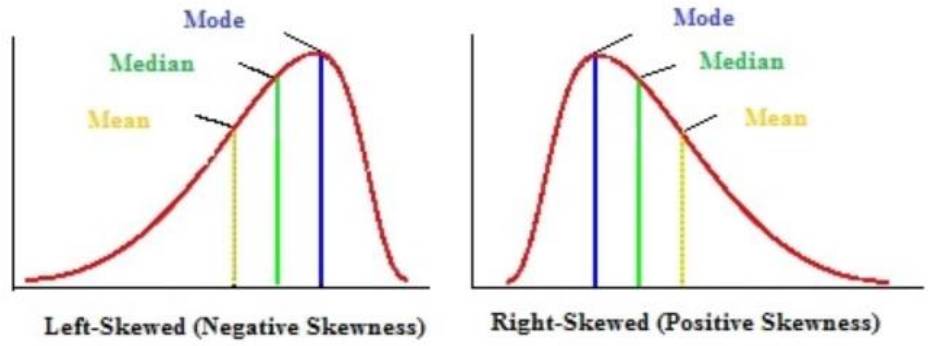
```
library(dplyr)
#library(tidyr)
agehist<- Marriage %>%
  select(age)
ggplot(agehist,
       aes(age)) +
       geom_histogram(color="black", fill="white")
```

Box plots

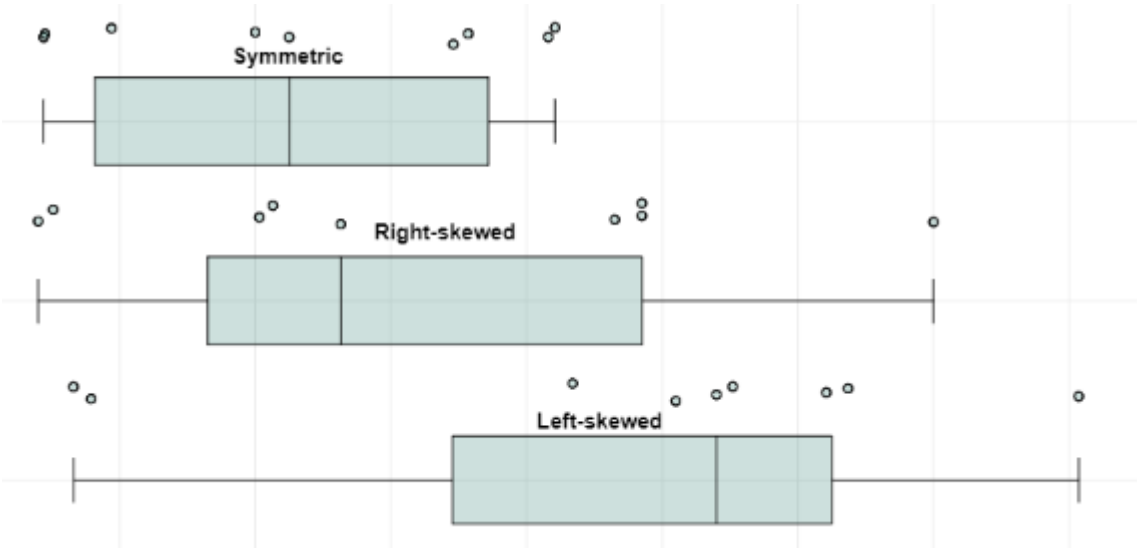
Box plots divide the data into sections that each contain approximately 25% of the data in that set.



The smallest value and largest value are found at the end of the 'whiskers' and are useful for providing a visual indicator regarding the spread of scores. In a box plot, an outlier is defined as a data point that is located outside the whiskers of the box plot.
The box plot shape will show whether the data are symmetrically distributed or skewed.



When the median is in the middle of the box, and the whiskers are about the same on both sides of the box, then the distribution is symmetric.

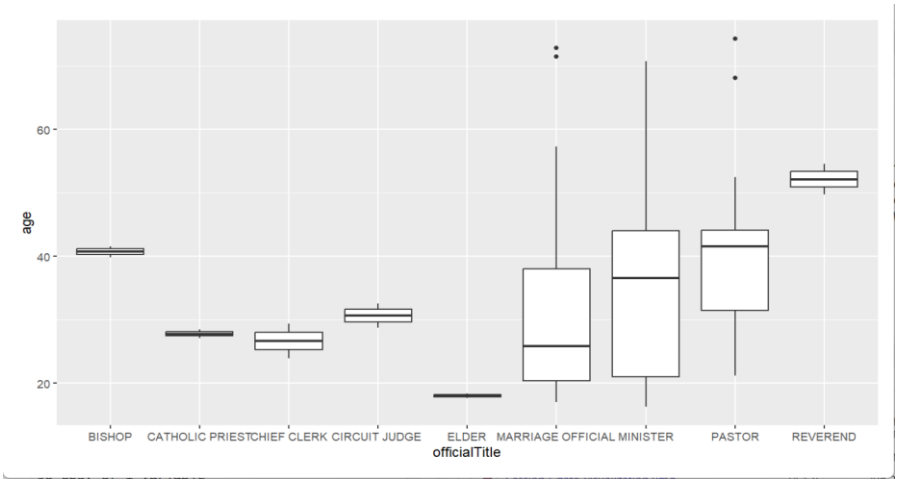


When the median is closer to the bottom of the box, and if the whisker is shorter on the lower end of the box, then the distribution is positively skewed (skewed right).
When the median is closer to the top of the box, and if the whisker is shorter on the upper end of the box, then the distribution is negatively skewed (skewed left).

Drawing a boxplot

Draw a boxplot for the variable `age` (age of the person (in years) against `officialTitle` variable in `Marriage` dataset and interpret results.

```
agebox<- Marriage %>%
  select(age, officialTitle)
ggplot(agebox,
       aes(x= officialTitle, y= age)) +
       geom_boxplot()
```



Refer https://pushpa-wijekoon.netlify.app/post/data-visualization/drawing-box-plots/ to understand more details for drawing boxplots.

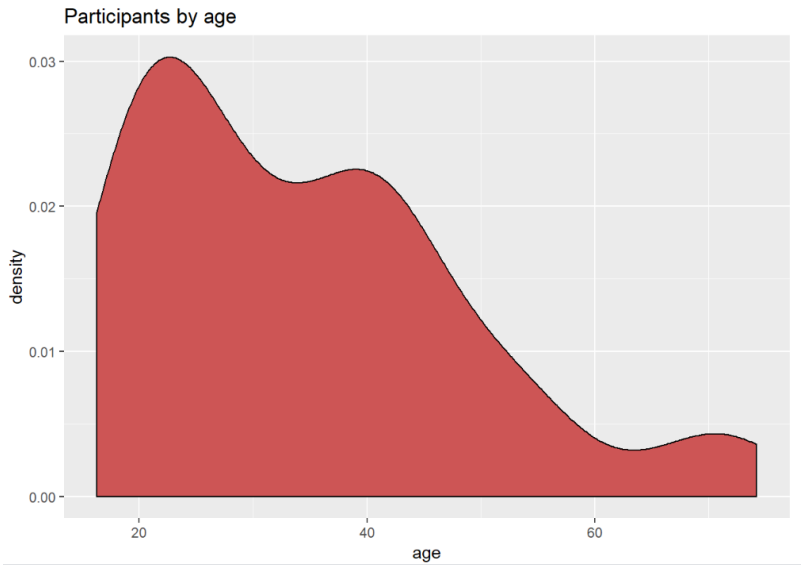
Kernel Density plot

An alternative to a histogram is the kernel density plot. Kernel density estimation is a nonparametric method for estimating the probability density function of a continuous random variable. Here, we are trying to draw a smoothed histogram, where the area under the curve equals one.

Drawing a kernel density plot

Using the Marriage dataset, let's draw the kernel density plot of the wedding participants by ages.

```
# Create a kernel density plot of age
ggplot(Marriage, aes(x = age)) +
  geom_density(fill = "indianred3") +
  labs(title = "Participants by age")
```



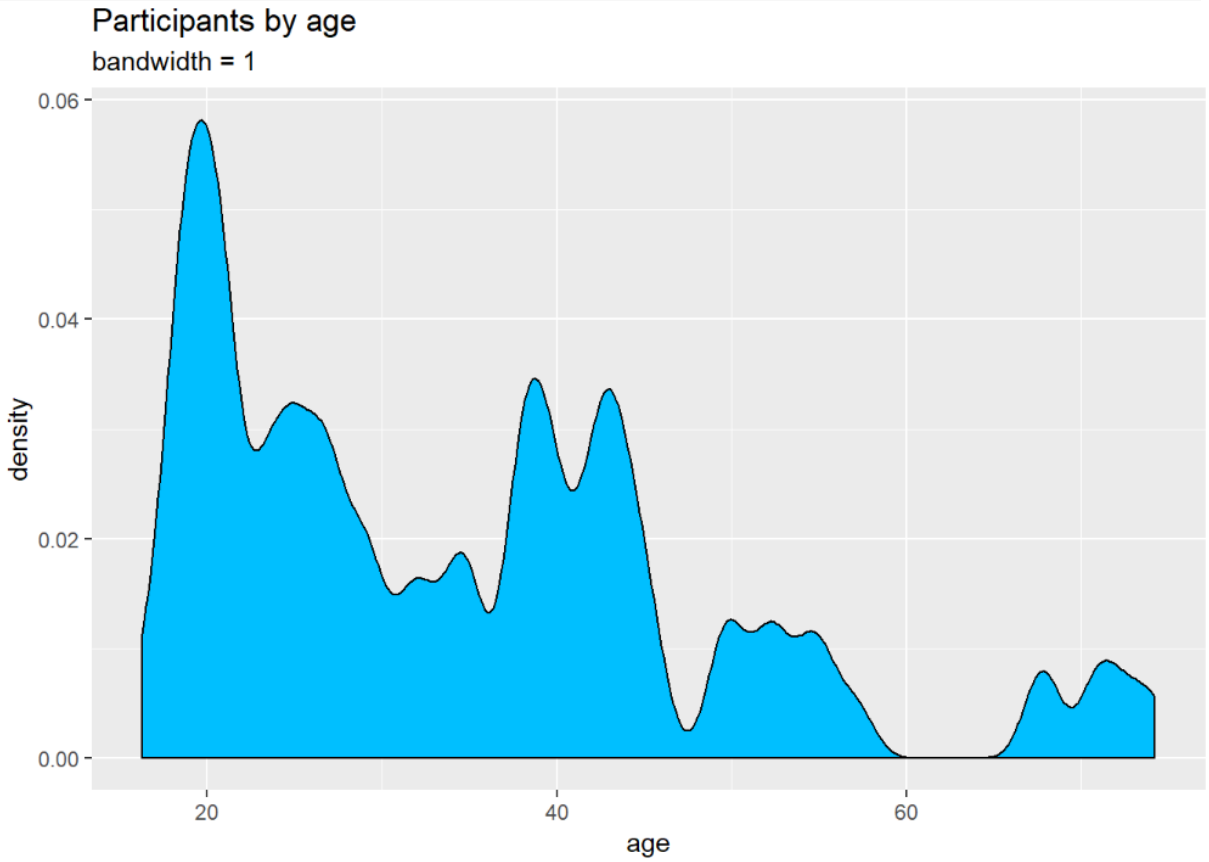
The graph shows the distribution of the scores. The degree of smoothness of kernel density plot is controlled by the bandwidth parameter `bw`. To find the default value for a particular variable, use the `bw.nrd0` function. Values that are larger will result in more smoothing, while values that are smaller will produce less smoothing.

```
# default bandwidth for the age variable
bw.nrd0(Marriage$age)

## [1] 5.181946
```

In this example, the default bandwidth for age is 5.18. Choosing a value of 1 resulted in less smoothing.

```
# Create a kernel density plot of age with bandwidth 1
ggplot(Marriage, aes(x = age)) +
  geom_density(fill = "deepskyblue",
               bw = 1) +
  labs(title = "Participants by age",
       subtitle = "bandwidth = 1")
```



Kernel density plots allow you to easily see which scores are most frequent and which are relatively rare. However, it may be difficult to explain the meaning of the *y*-axis to a non-statistician.

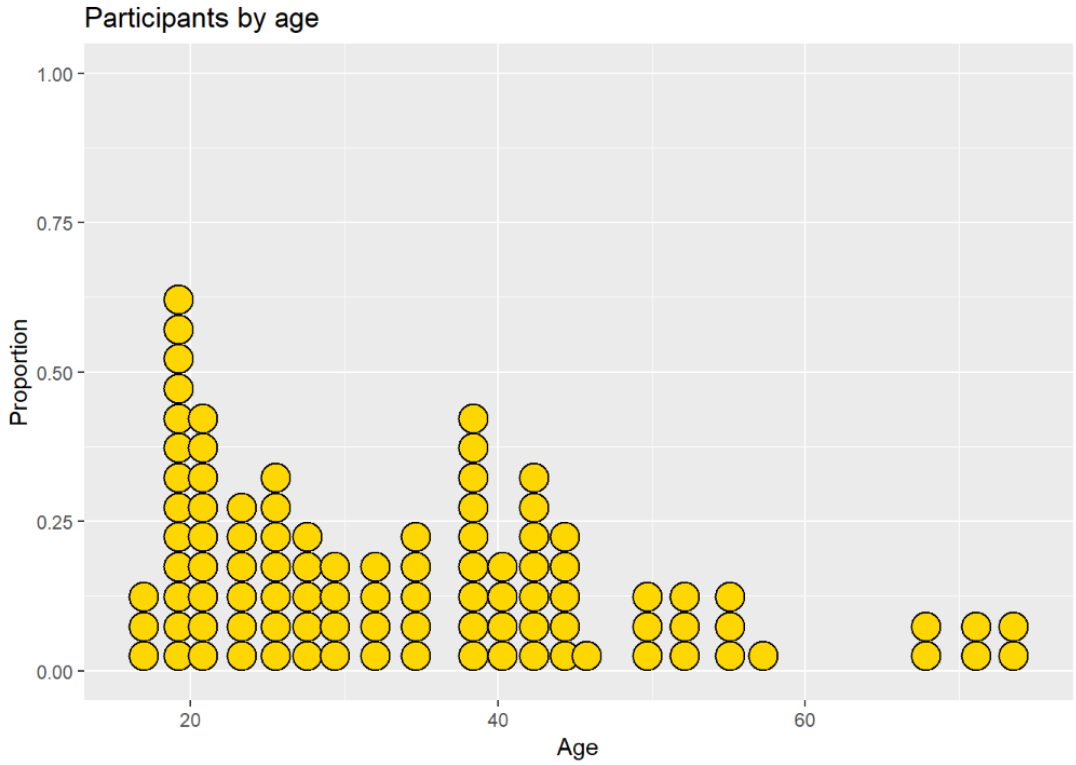
Dot plot

Another alternative to the histogram is the dot chart. As in the histogram, the quantitative variable is divided into bins. Although in a histogram, bars represent frequencies, in the dot plot each observation is represented by a dot. The dot plot works best when the number of observations is small.

Drawing a dot plot

Plot the age distribution of the Marriage dataset using a dot plot.

```
ggplot(Marriage, aes(x = age)) +
  geom_dotplot(fill = "gold",
               color = "black") +
  labs(title = "Participants by age",
       y = "Proportion",
       x = "Age")
```



To interpret dot plots, identify the peaks, which are the bins that have the most dots.



Right-skewed



Left-skewed

6.2 Bivariate Graphs

Bivariate graphs display the relationship between two variables. The type of graph will depend on the measurement level of the variables (categorical or quantitative).

Using `knitr::opts_chunk$set()` code chunk, you can setup global options which apply to every chunk in your .rmd document. When you Knit the document this will treat each option that you pass to `knitr::opts_chunk$set` as a global default that can be overwritten in individual chunk headers.

Add this as the first code chunk of your document. For example, we now setup the .rmd file as below:

```
```{r setup, include=FALSE}
require(knitr)
knitr::opts_chunk$set(echo = TRUE,
                      warning=FALSE,
                      message=FALSE,
                      fig.align='center')
```
```

6.2.1 Categorical vs. Categorical graphs

When plotting the relationship between two categorical variables, stacked, grouped, or segmented bar charts are typically used. A less common approach is the mosaic chart.

Example 5: mpg dataset

The mpg dataset from the `ggplot2` package, contains fuel economy data for 38 popular models of car, for the years 1999 and 2008. This data set includes 234 observations for 11 variables. The variables are

Variable	Description	Type
manufacturer	Manufacturer name	chr
model	Model name	chr
displ	Engine displacement in litres	dbl
year	Year of manufacture	int

cyl	Number of cylinders	int
trans	Type of tranmission	chr
dry	The type of drive train, where f = front-wheel drive, r = rear wheel drive, 4 = 4wd	chr
cty	City miles per gallon	int
hwy	Highway miles per gallon	int
fl	Fuel type	chr
class	Type of the car	chr

Then, we access the dataset and get the relevant information using

```
data(mpg, package="ggplot2")
library(dplyr)
glimpse(mpg)
summary(mpg)
```

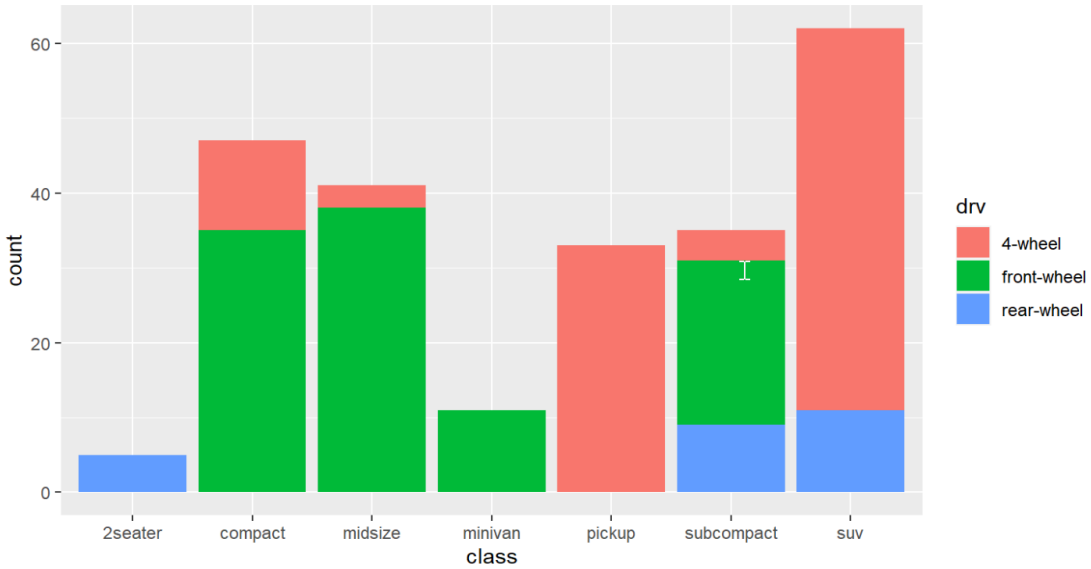
6.2.1.1 Bar Charts

Now, we select two categorical variables **`class`** and **`dry`** variables, and plot the relationship between automobile class and drive type (front-wheel, rear-wheel, or 4-wheel drive).

Drawing a stacked bar chart

First, we draw a **stacked bar chart** using the following R codes.

```
library(ggplot2)
ggplot(mpg,
       aes(x = class,
           fill = drv)) +
  geom_bar(position = "stack")+
scale_fill_discrete(labels = c("4-wheel", "front-wheel", "rear-wheel"))
```

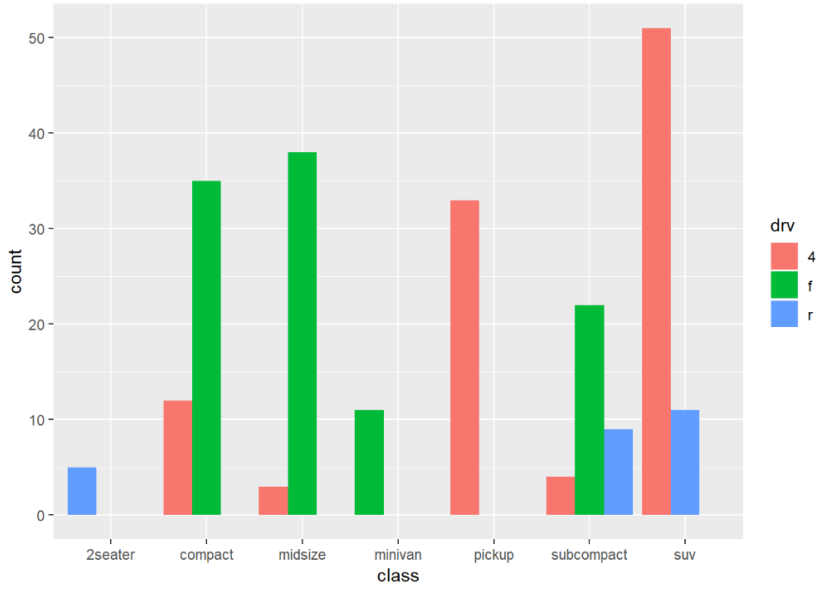


According to the chart, the most common vehicle type is the SUV. All 2seater cars are rear wheel drive, and most of the SUVs are 4-wheel drive. All minivans are front wheel drive, and all pickups are four wheel.

Drawing a grouped bar chart

Now, we draw a **grouped bar chart**. To create a grouped bar plot, we have to change the position as `position = "dodge"`. ggplot2 reference guide is here: https://ggplot2.tidyverse.org/reference/.

```
library(ggplot2)
ggplot(mpg,
       aes(x = class,
           fill = drv)) +
  geom_bar(position = position_dodge(preserve = "single"))
```

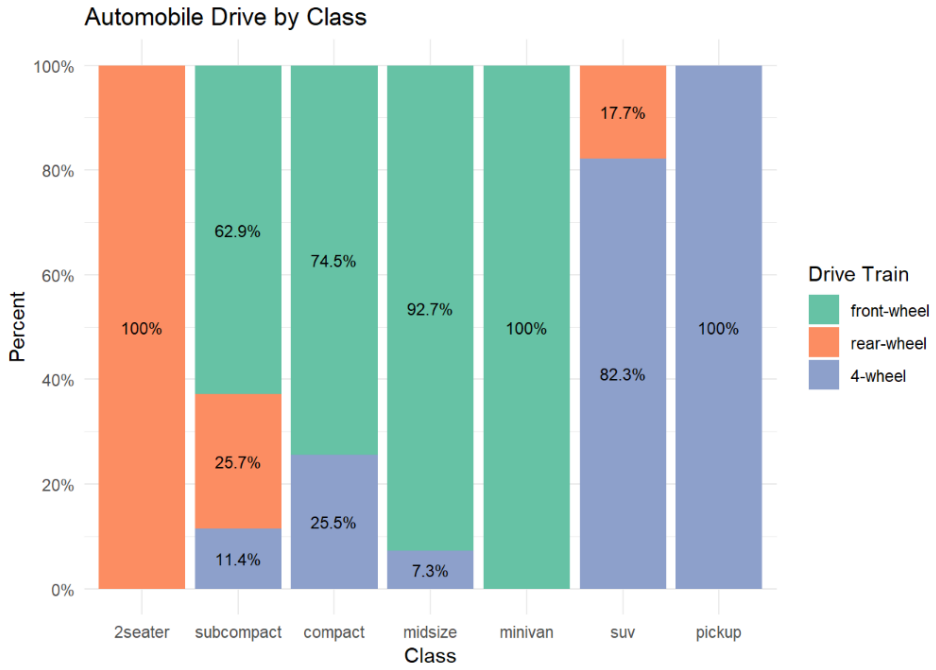


Drawing a segmented bar plot

Next, we draw a segmented bar plot which is a stacked bar plot where each bar represents 100 percent.

```
library(dplyr)
plotdata <- mpg %>%
  group_by(class, drv) %>%
  summarize(n = n()) %>%
  mutate(pct = n/sum(n),
         lbl = scales::percent(pct))
ggplot(plotdata,
   aes(x = factor(class,
         levels = c("2seater", "subcompact","compact", "midsize",
                                "minivan", "suv", "pickup")),
       y = pct,
       fill = factor(drv,
```

```
            levels = c("f", "r", "4"),
            labels = c("front-wheel","rear-wheel","4-wheel")))) +
  geom_bar(stat = "identity",
           position = "fill") +
  scale_y_continuous(breaks = seq(0, 1, .2)) +
  geom_text(aes(label = lbl),
            size = 3,
            position = position_stack(vjust = 0.5)) +
  scale_fill_brewer(palette = "Set2") +
  labs(y = "Percent",
       fill = "Drive Train",
       x = "Class",
       title = "Automobile Drive by Class") +
  theme_minimal()
```



6.2.1.2 Mosaic charts

Mosaic charts display the **relationship between categorical variables using rectangles** whose areas represent the proportion of cases for any given combination of levels. The color of the tiles can also indicate the degree of the relationship among the variables.

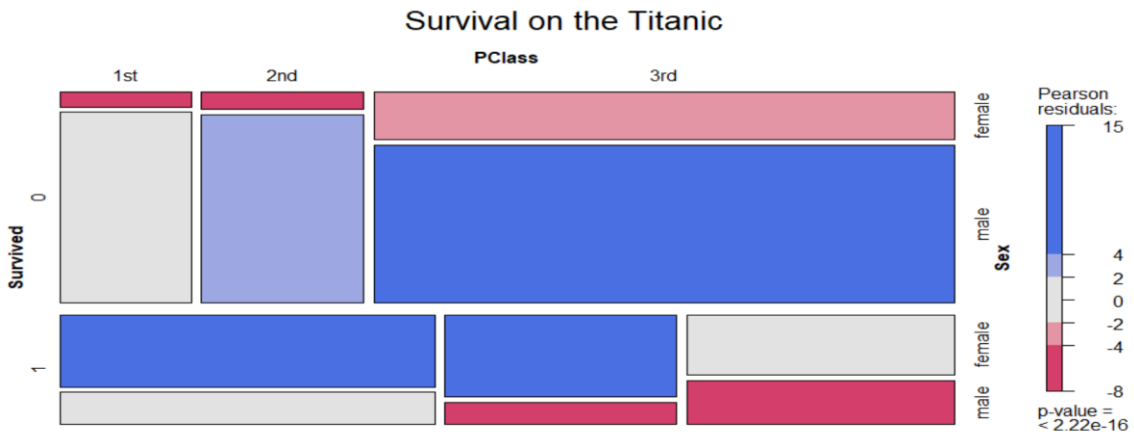
Drawing a Mosaic chart

In this example we use the Titanic data set which can be downloaded from https://github.com/datasciencedojo/datasets/blob/master/titanic.csv. First, we read the data.

```
library(readr)
titanic <- read_csv("Titanic.csv")
```

Now, we try to visualize the relationship between the three categorical variables, Survived (0 = No, 1 = Yes), PClass ((1 = 1st, 2 = 2nd, 3 = 3rd), and Sex (male, female) using the code below:

```
tbl <- xtabs(~Survived + Class + Sex, titanic)
ftable(tbl)
library(vcd)
mosaic(tbl, main ="Survival on the Titanic", shade = TRUE, legend = TRUE)
```



Here, we can see that more 3rd class male passengers were died, and more female passengers were survived.

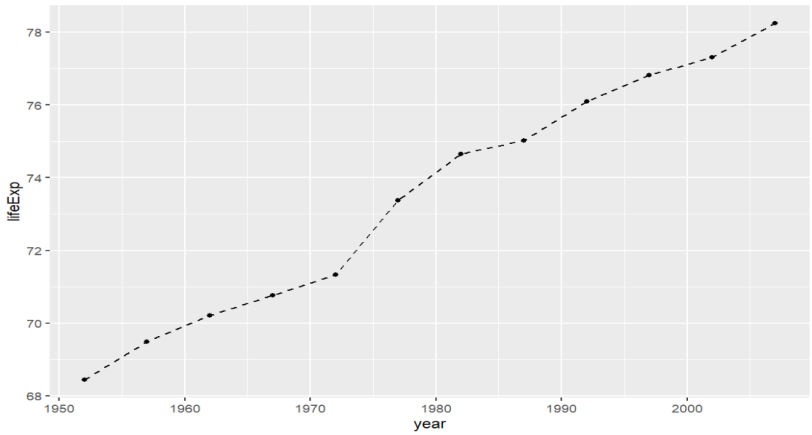
6.2.2. Quantitative vs. Quantitative

To represent the relationship between two quantitative variables, the usual practice is to use scatterplots and line graphs.

Drawing a Line graph

Here we use the gapminder dataset from the `gapminder` package. This data set contains longitudinal data (1952-2007) on life expectancy, GDP per capita, and population for 142 countries. Line graph which displays the relationship between time (*year*) and life expectancy (*lifeExp*) in the United States between 1952 and 2007.

```
data(gapminder, package="gapminder")
# subset Asian countries in 2007
library(dplyr)
plotdata <- gapminder %>%
  filter(continent == "Americas" & country=="United States")
library(ggplot2)
ggplot(plotdata, aes(x=year, y=lifeExp)) +
geom_line(linetype = "dashed")+
geom_point()
```



6.2.3 Categorical vs. Quantitative

When plotting the relationship between a categorical variable and a quantitative variable, we can use bar charts using summary statistics, grouped kernel density plots, side-by-side box plots, mean/sem plots, ridgeline plots, and Cleveland plots.

When drawing bar charts, we can use summary statistics (e.g., means or medians) on a quantitative variable for each level of a categorical variable.

Drawing a bar chart for categorical variables

The following graph displays the mean salary for a sample of university professors by their academic rank. The data set is attached in `"carData" package.`

```
data(Salaries, package="carData")
# calculate mean salary for each rank
library(dplyr)
plotdata <- Salaries %>%
  group_by(rank) %>%
  summarize(mean_salary = mean(salary))

library(scales)
ggplot(plotdata,
  aes(x = factor(rank,
    labels =
c("Assistant\nProfessor","Associate\nProfessor","Full\nProfessor")),
        y = mean_salary)) +
  geom_bar(stat = "identity", fill = "cornflowerblue") +
  geom_text(aes(label = dollar(mean_salary)), vjust = -0.25) +
  scale_y_continuous(breaks = seq(0, 130000, 20000),label = dollar) +
       labs(title = "Mean Salary by Rank",
       subtitle = "9-month academic salary for 2008-2009", x = "", y = "")
```

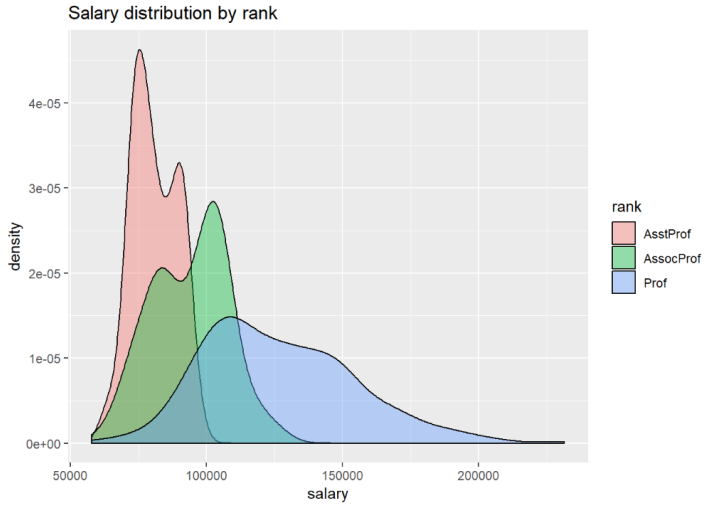


Grouped kernel density plots

To compare groups on a numeric variable, we can superimpose kernel density plots in a single graph.

Drawing a grouped kernel density plot

```
ggplot(Salaries,
       aes(x = salary,
           fill = rank)) +
  geom_density(alpha = 0.4) +
  labs(title = "Salary distribution by rank")
```



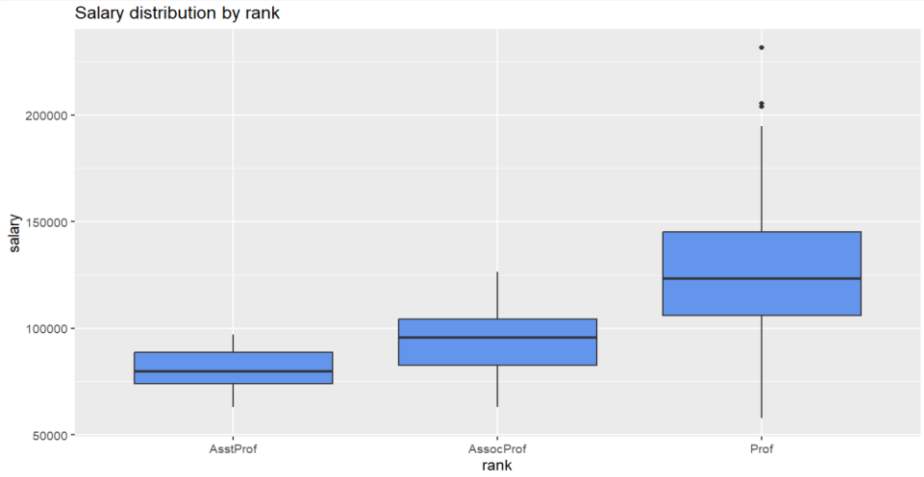
To make the density plots partially transparent we can use the `alpha` option. Then, we can clearly see what is happening under the overlaps. The range of alpha is from 0 (transparent) to 1 (opaque). According to the graph, the salary goes up with rank. However, the salary range for full professors is *very* wide.

Side-by-side box plots

Side-by-side box plots are especially useful for comparing the levels of a categorical variable on a numerical variable.

Drawing side-by-side box plots

```
ggplot(Salaries,
       aes(x = rank,
           y = salary)) +
  geom_boxplot(fill = "cornflowerblue") +
    labs(title = "Salary distribution by rank")
```



Ridgeline plots

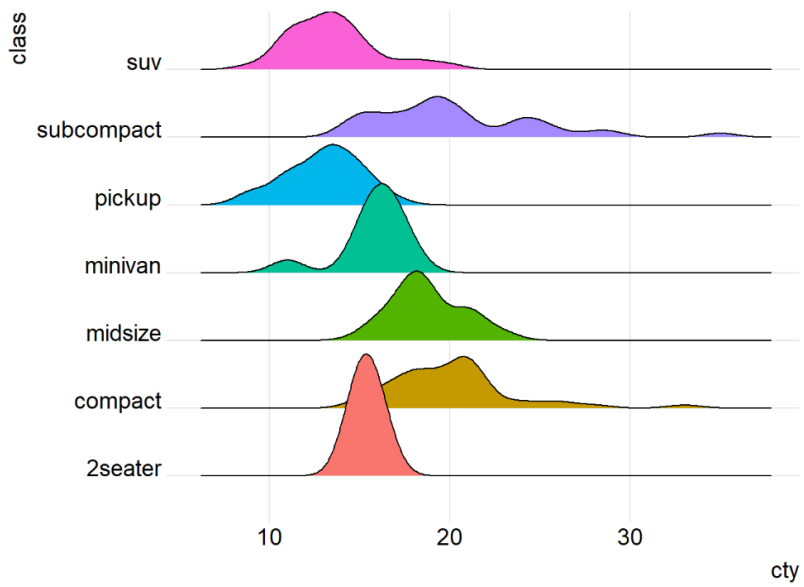
A ridgeline plot (also called a joyplot) is similar to kernel density plots. To create ridgeline plots we need the `ggridges` package.

Drawing a Ridgeline plot

To draw a ridgeline plot, we use the mpg dataset from the `ggplot2` package. Here we plot the distribution of city driving miles per gallon by car class.

```
library(ggplot2)
library(ggridges)

ggplot(mpg,
       aes(x = cty,
           y = class,
           fill = class)) +
  geom_density_ridges() +
  theme_ridges() +
  labs("Highway mileage by auto class") +
  theme(legend.position = "none")
```



Here you can see that pickup trucks have the poorest mileage, while the distribution of mileage for subcompacts and compact cars have higher values. However, there is a very wide range of gas mileage scores for these smaller cars.

Mean/SEM plots

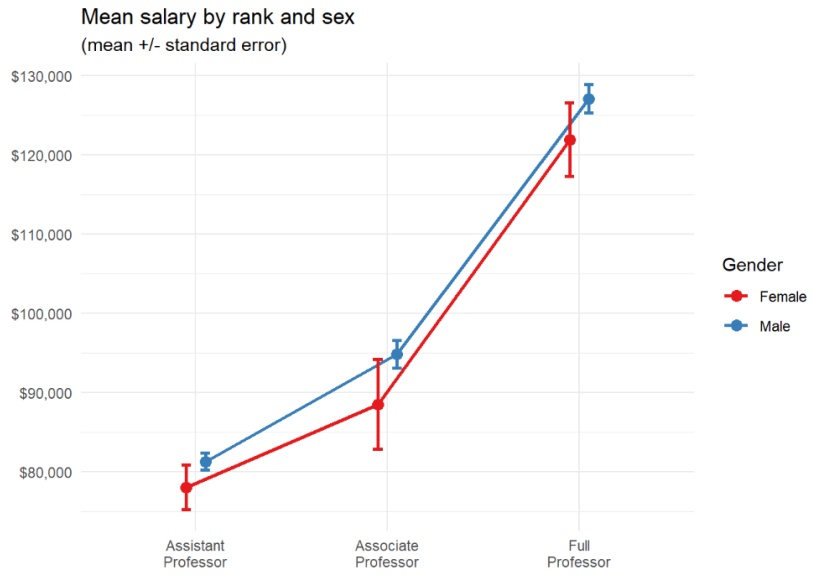
Mean/SEM plots are used to compare groups on a numeric variable with error bars. Error bars can represent standard deviations, standard error of the mean, or confidence intervals.

Drawing a Mean/SEM plot

Here we use "Salaries" data set in "carData" package. Now, we draw a Mean/SEM plot to compare salary across rank and sex variables by using means and standard errors.

```
# Calculate means, standard errors and confidence intervals by rank and sex
ibrary(dplyr)
plotdata <- Salaries %>%
  group_by(rank, sex) %>%
  summarize(n = n(),
            mean = mean(salary),
            sd = sd(salary),
            se = sd/sqrt(n),
            ci = qt(0.975, df = n - 1) * sd / sqrt(n))
# To avoid error bars overlap,dodge the horizontal position
pd <- position_dodge(0.2)
ggplot(plotdata,
   aes(x = factor(rank,
     labels = c("Assistant\nProfessor","Associate\nProfessor",
                                "Full\nProfessor")),
           y = mean,
           group=sex,
           color=sex)) +
  geom_point(position=pd, size = 3) +
  geom_line(position = pd, size = 1) +
```

```
  geom_errorbar(aes(ymin = mean - se, ymax = mean + se),
                width = .1, position = pd, size = 1) +
  scale_y_continuous(label = scales::dollar) +
  scale_color_brewer(palette="Set1") +
  theme_minimal() +
  labs(title = "Mean salary by rank and sex",
       subtitle = "(mean +/- standard error)",x = "",y = "",color =
"Gender")
```



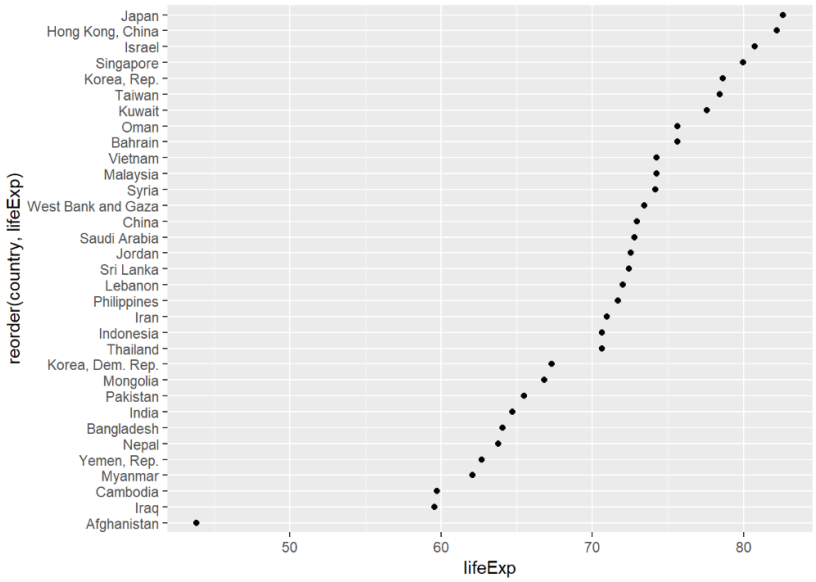
Cleveland Dot Charts

Cleveland plots are useful when we want to compare numeric statistics for a large number of groups.

Drawing Cleveland Dot Chart

Here we again use the gapminder dataset from the `gapminder` package. In this example we compare the 2007 life expectancy for Asian countries.

```
data(gapminder, package="gapminder")
# subset Asian countries in 2007
library(dplyr)
plotdata <- gapminder %>%
  filter(continent == "Asia" & year == 2007)
ggplot(plotdata,
       aes(x=lifeExp, y=reorder(country, lifeExp))) +
  geom_point()
```



6.3 Multivariate graphs

Multivariate graphs are used to display the relationships among three or more variables. To accommodate multiple variables, we have to group or facet the variables.

In grouping, first select two variables as *x* and *y* variables. Then, the additional variables are mapped to other visual characteristics such as color, shape, size, line type, and transparency. In this way, we can plot multiple variables in a single graph.

6.3.1 Drawing a multivariate scatter plot

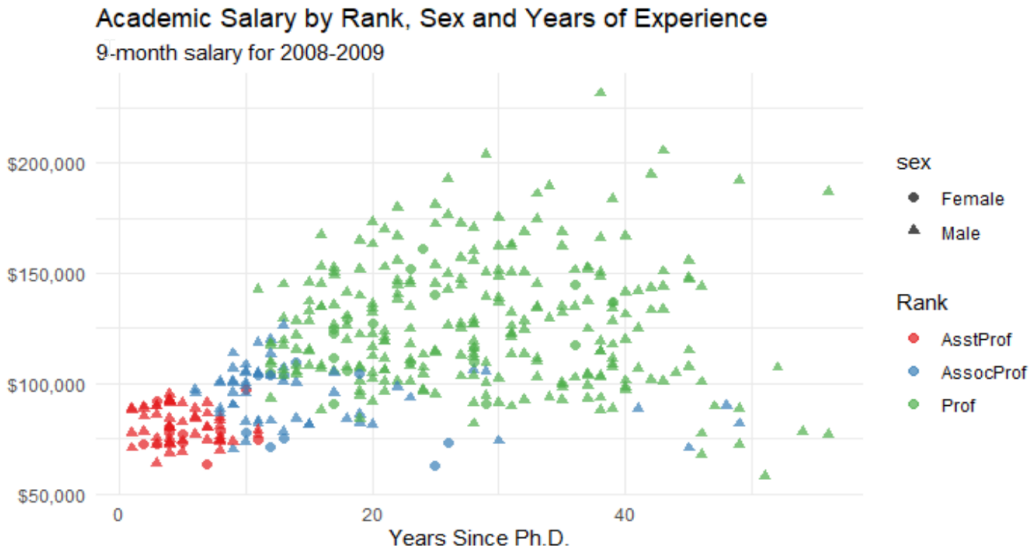
Again, we use "Salaries" data set in "carData" package. Plot a multivariate graph to display the relationship between the variables *yrs.since.phd* and *salary*, and add the rank of the professor using color, the gender of professor using the shape of the points to indicate sex, and add transparency to make the individual points clearer.

```
library(ggplot2)
data(Salaries, package="carData")
ggplot(Salaries,
       aes(x = yrs.since.phd,
           y = salary,
           color = rank, shape = sex)) +
  geom_point(size = 2, alpha = .7) +
labs(x = "Years Since Ph.D.",
       title = "Academic Salary by Rank, Sex and Years of Experience",
       subtitle = "9-month salary for 2008-2009",
       y = "",
       color = "Rank") +
  scale_y_continuous(label = scales::dollar) +
  scale_color_brewer(palette = "Set1") +
  theme_minimal()
```

`scale_x_continuous()` and `scale_y_continuous()` are the default scales for continuous x and y aesthetics. There are three variants that set the `trans` argument for commonly used transformations:

`scale_*_log10()`, `scale_*_sqrt()` and `scale_*_reverse()`.

Refer: https://ggplot2.tidyverse.org/reference/scale_continuous.html

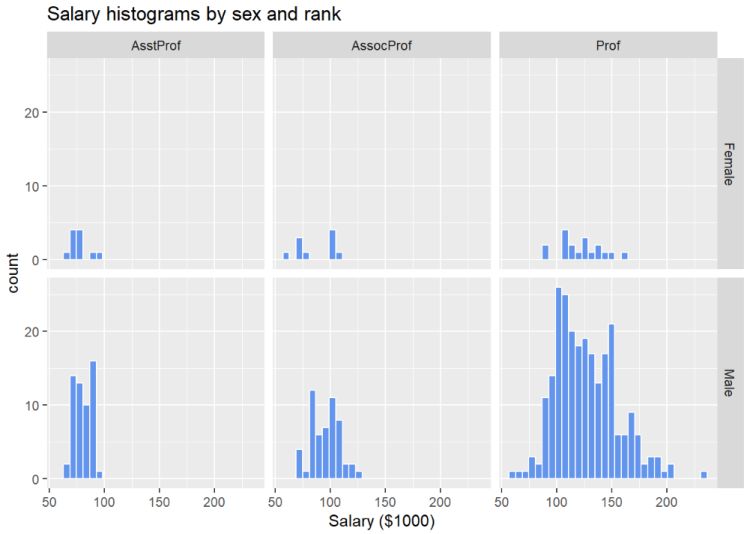


When drawing **a multivariable graph** using **faceting, it** consists of several separate plots or *small multiples*, one for each level of a third variable, or combination of variables.

6.3.2 Drawing histograms for multivariate data

Using the "Salaries" data set, draw a histogram for the variable x=salary/1000 by sex and rank.

```
ggplot(Salaries, aes(x = salary / 1000)) +
  geom_histogram(color = "white",
                 fill = "cornflowerblue") +
  facet_grid(sex ~ rank) +
  labs(title = "Salary histograms by sex and rank",
       x = "Salary ($1000)")
```



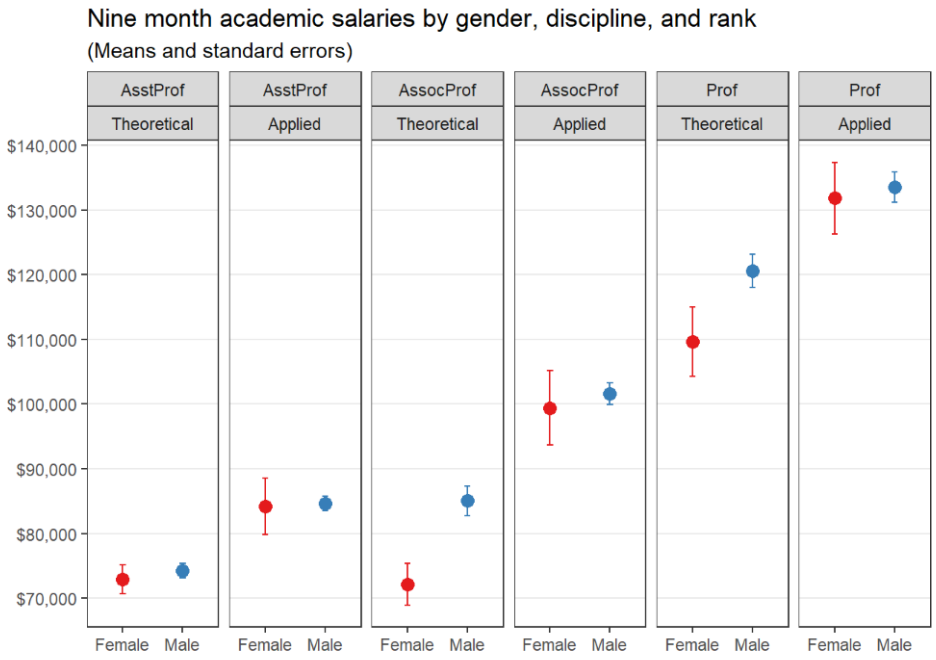
6.3.3 Mean/SE plots for multivariate data

We can also combine grouping and faceting. Now, we draw Mean/SE plots and faceting to compare the salaries of male and female professors, within rank and discipline. To distinguish sex, we use colour, and use faceting for rank by different discipline.

Drawing Mean/SE plots for multivariate data

```
library(dplyr)
plotdata <- Salaries %>%
  group_by(sex, rank, discipline) %>%
  summarize(n = n(),
            mean = mean(salary), sd = sd(salary), se = sd / sqrt(n))

plotdata$discipline <- factor(plotdata$discipline,
                              labels = c("Theoretical", "Applied"))
ggplot(plotdata,
       aes(x = sex, y = mean, color = sex)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean - se, ymax = mean + se), width = .1) +
  scale_y_continuous(breaks = seq(70000, 140000, 10000),
                     label = scales::dollar) +
  facet_grid(. ~ rank + discipline) +
  theme_bw() +
  theme(legend.position = "none",
        panel.grid.major.x = element_blank(),
        panel.grid.minor.y = element_blank()) +
  labs(x="",
       y="",
       title="Nine month academic salaries by gender, discipline, and
rank",
       subtitle = "(Means and standard errors)") +
  scale_color_brewer(palette="Set1")
```



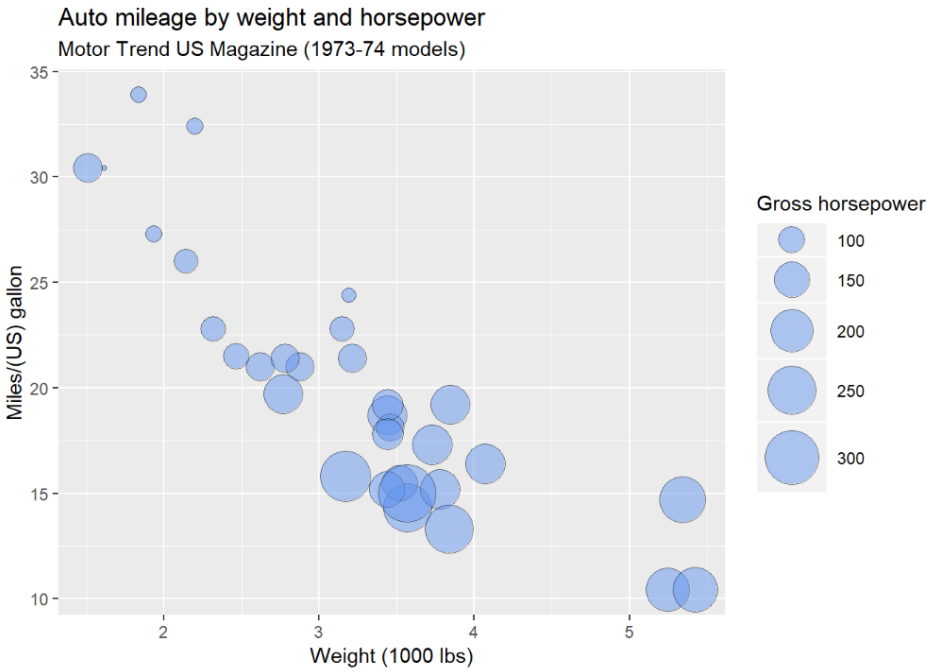
6.3.4 Bubble chart for multivariate data

A bubble chart is also a scatterplot where the point size is proportional to the values of a third quantitative variable.

Drawing a Bubble chart for multivariate data

Use "mtcars" data set available in "datasets" R package and draw a bubble chart for car weight vs. mileage, and use point size to represent horsepower.

```
data(mtcars, package="datasets")
library(ggplot2)
ggplot(mtcars,
       aes(x = wt, y = mpg, size = hp)) +
  geom_point(alpha = .5,
             fill="cornflowerblue", color="black", shape=21) +
  scale_size_continuous(range = c(1, 14)) +
  labs(title = "Auto mileage by weight and horsepower",
       subtitle = "Motor Trend US Magazine (1973-74 models)",
       x = "Weight (1000 lbs)",
       y = "Miles/(US) gallon",
       size = "Gross horsepower")
```



According to this graph, miles per gallon decreases with increased car weight and horsepower.

6.3.5 Correlation plots

We can draw correlation plots to visualize the pairwise relationships between a set of quantitative variables by displaying their correlations using color or shading.

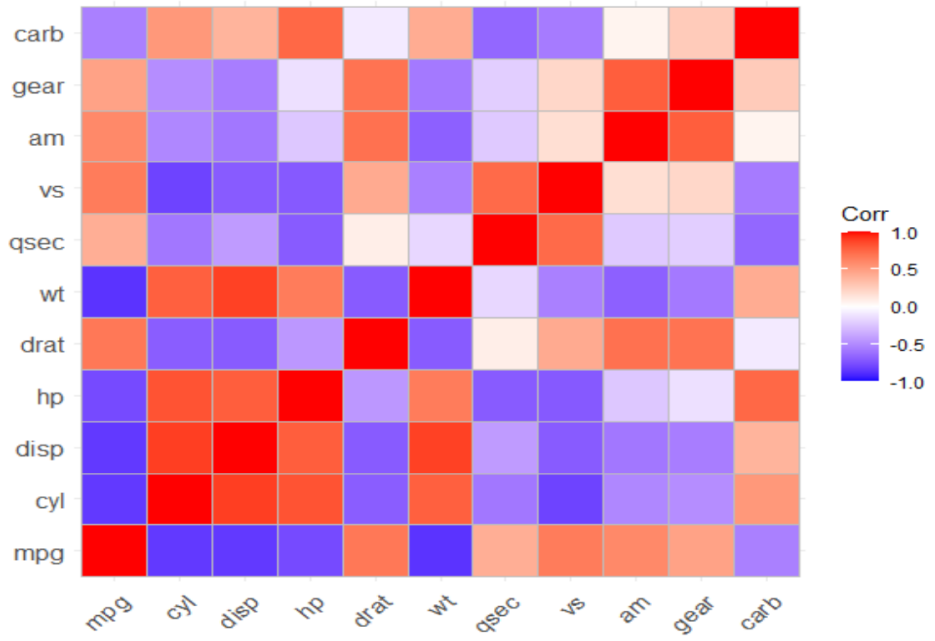
Drawing a Correlation plots for multivariate data

Draw a correlation plot for the numeric variables in "mtcars" data set available in `"datasets" R package`. The `ggcorrplot` function in the `ggcorrplot package` can be used to visualize these correlations.

```
data(mtcars, package="datasets")
# Select numeric variables
df <- dplyr::select_if(mtcars, is.numeric)

# Calulate the correlations by discarding the entire row if an NA is
present.
r <- cor(df, use="complete.obs")
round(r,2)
library(ggplot2)
library(ggcorrplot)
ggcorrplot(r)
```

```
More details
Refer http://www.sthda.com/english/wiki/ggcorrplot-visualization-of-a-
correlation-matrix-using-ggplot2
```



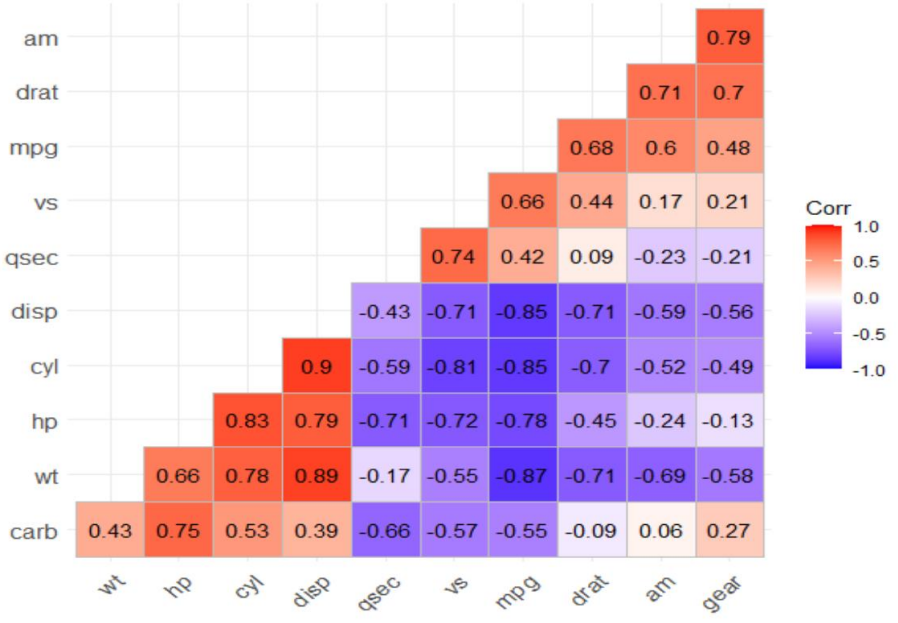
The above plot can be further improved by customizing the following options in the `ggcorrplot` function.

`hc.order = TRUE` reorders the variables, placing variables with similar correlation patterns together.

`type = "lower"` plots the lower portion of the correlation matrix.

`lab = TRUE` overlays the correlation coefficients (as text) on the plot.

```
ggcorrplot(r, hc.order = TRUE, type = "lower", lab = TRUE)
```



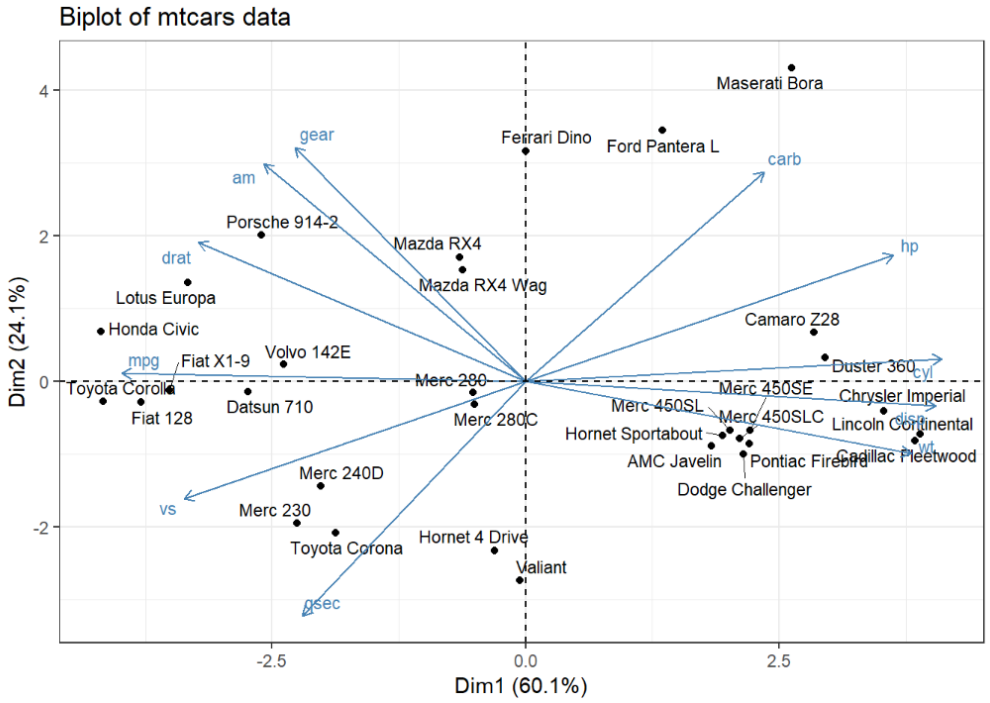
6.3.6 Biplots

Biplots graphically displays the results of the Principal Component Analysis (PCA). Each variable of a principal component has an associated arrow. Arrows for each variable point in the direction of increasing values of that variable. The horizontal and vertical axes of the plot are Principal Components.

Drawing Biplots for multivariate data

Let us consider the `mtcars` data set and draw a biplot using the first two Principal Components.

```
data(mtcars, package="datasets")
# fit a principal components model
fit <- prcomp(x = mtcars, center = TRUE, scale = TRUE)
# Install factoextra package
library(factoextra)
fviz_pca(fit,
         repel = TRUE,
         labelsize = 3) +
  theme_bw() +
  labs(title = "Biplot of mtcars data")
```



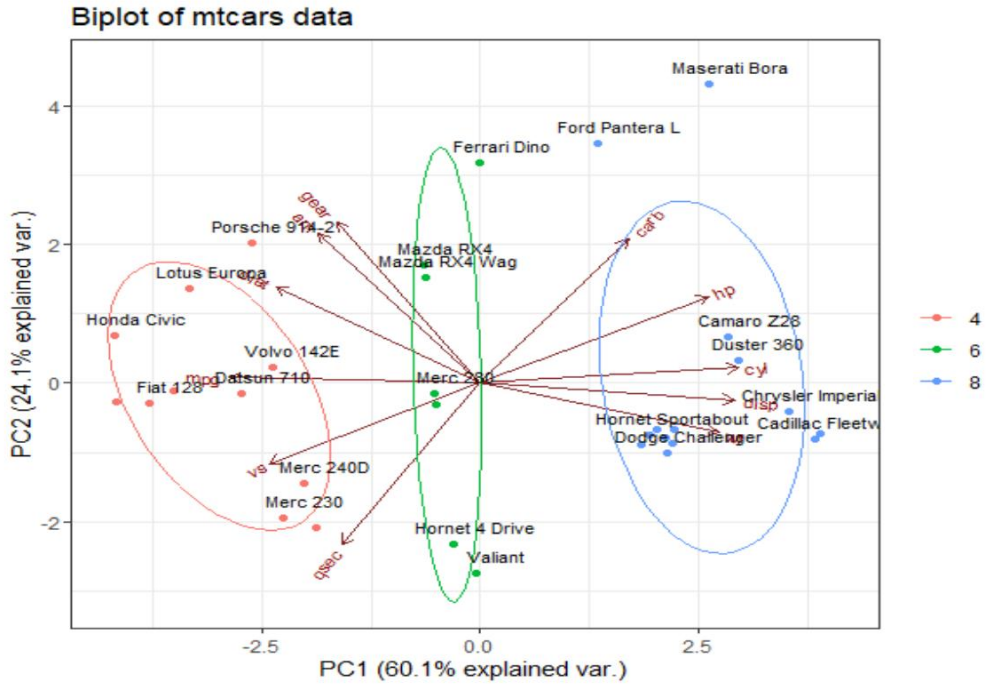
In this graph, the first PC accounts for 60% of the variability in the original data, and the second PC accounts for 24%. Then, both PC's account for 84% of the variability in the original $p = 11$ variables. In the plot, both the observations (car types) and variables (car characteristics) are plotted in the same graph.

- Points represent observations, and if they are closer then, their values are similar on the original set of variables. Accordingly, the *Toyota Corolla* and *Honda Civic* are similar to each other, as are the *Chrysler Imperial* and *Liconln Continental*. However, the *Toyota Corolla* is quite different from the *Lincoln Continental*.

- The vectors (arrows) represent variables, and the angle between vectors are proportional to the correlation between the variables. Smaller angles indicate stronger correlations. For example, *gear* and *am* are positively correlated, *gear* and *qsec* are uncorrelated (90 degrees angle), and *am* and *wt* are negatively correlated (angle greater than 90 degrees).
- The observations that are farthest along the direction of a variable's vector, have the highest values on that variable. For example, the *Toyoto Corolla* and *Honda Civic* have higher values on *mpg*. The *Toyota Corona* has a higher *qsec*. The *Duster 360* has more *cylinders*.

We can improve further the above plot by using `ggbiplot package` as below:

```
data(mtcars, package="datasets")
# fit a principal components model
fit <- prcomp(x = mtcars, center = TRUE, scale = TRUE)
source('https://raw.githubusercontent.com/vqv/ggbiplot/master/R/ggbiplot.r'
)
source('https://raw.githubusercontent.com/vqv/ggbiplot/master/R/ggscreeplot
.r')
data(mtcars, package="datasets")
# draw ellipses around the levels of cyl
cyl <- as.factor(mtcars[,2])
ggbiplot(fit, obs.scale = 1, group = cyl, ellipse = T) +
scale_color_discrete(name = '') +
geom_text(label=rownames(mtcars),
    nudge_x = 0.25, nudge_y = 0.25, check_overlap = T, size=3)+
labs(title = "Biplot of mtcars data") +
  theme_bw()
```



Based on this figure, we can clearly understand the relationships among the variables for each cylinder size. The 'qsec' arrow, points towards low values of PC1. This means the lower the value of PC1, the higher the qsec.

6.3.7 Scatterplot matrix

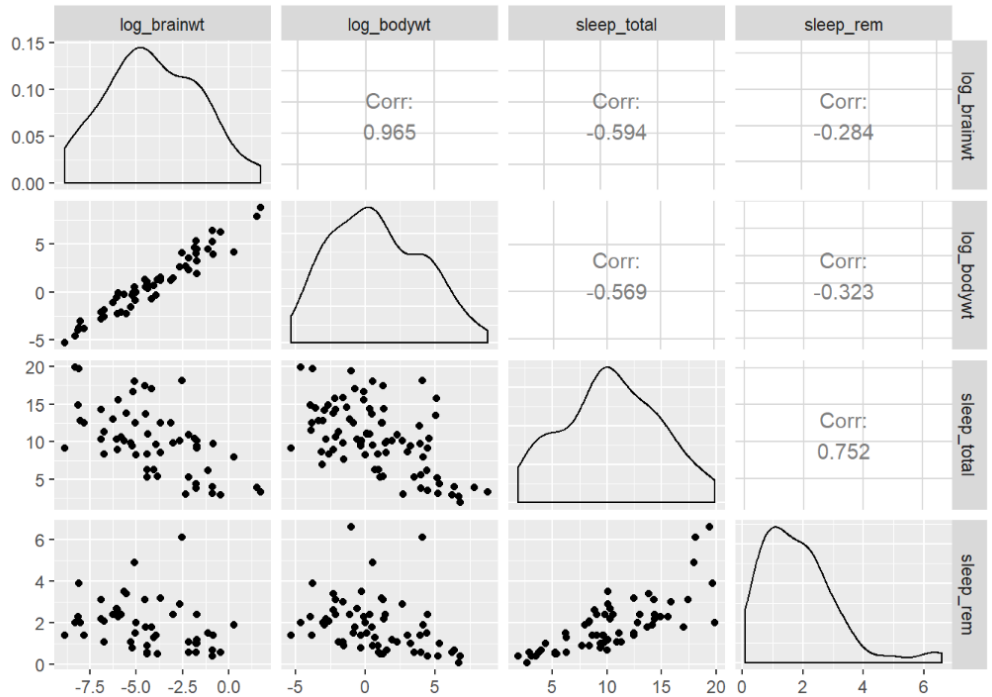
A scatterplot matrix is a collection of scatterplots organized as a grid. To create a scatterplot matrix, we can use the ggpairs function in the GGally package.

Drawing a Scatterplot matrix for multivariate data

In this example, we use "msleep" data set in "ggplot2" package. Plot a multivariate graph to display the relationships between mammal size and sleep characteristics. Before drawing the plot apply log transformation to brain weight and body weight variables since they are highly skewed.

```
data(msleep, package="ggplot2")
library(dplyr)
df <- msleep %>%
  mutate(log_brainwt = log(brainwt),
         log_bodywt = log(bodywt)) %>%
  select(log_brainwt, log_bodywt, sleep_total, sleep_rem)

# create a scatterplot matrix
library(GGally)
ggpairs(df)
```



According to the scatter plots we can clearly see that as brain weight increases, with total sleep time and time in REM sleep decrease. On the diagonal of this graph we have a kernel density plot for each variable. The cells above the diagonal contain the correlation coefficients.

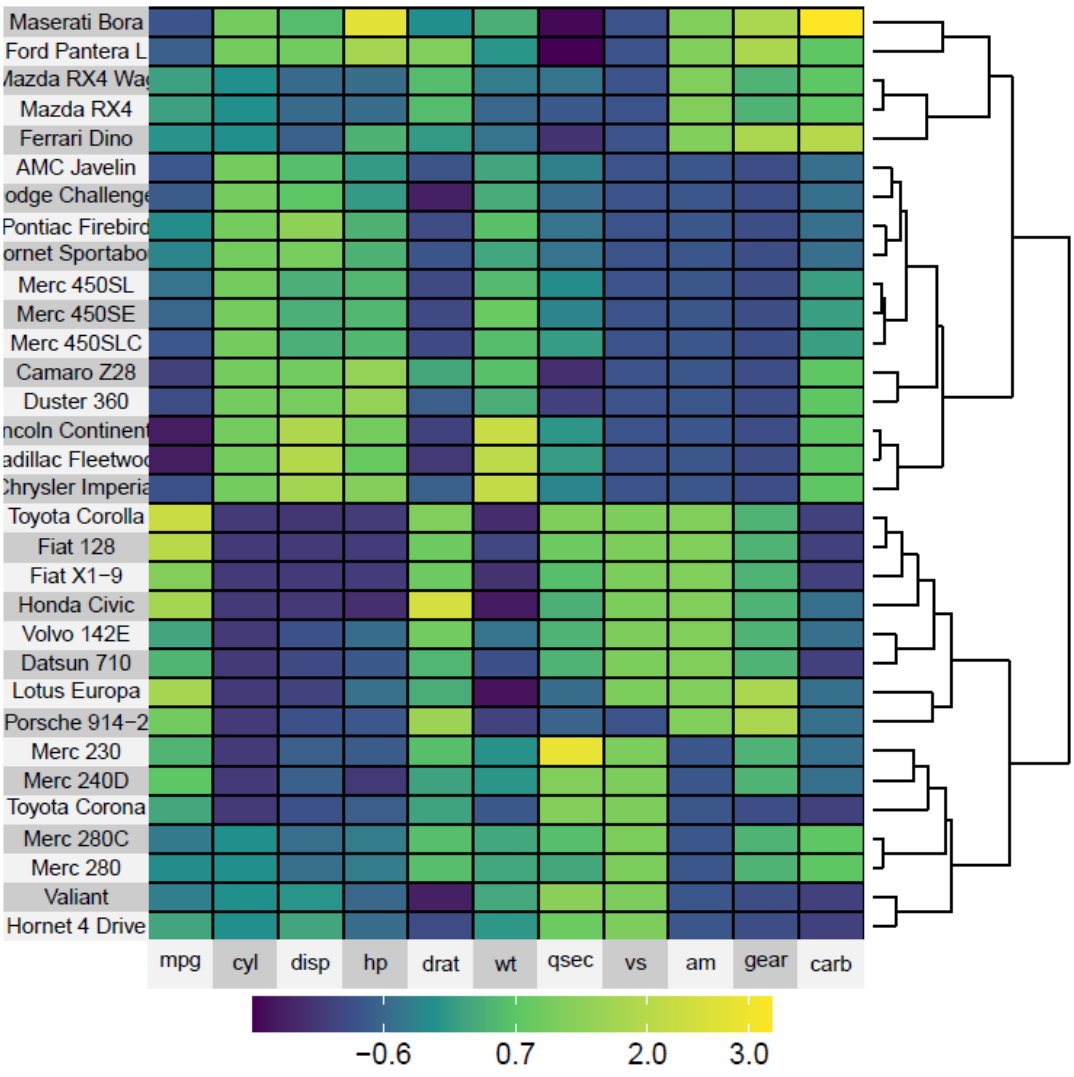
6.3.8 Heatmap

The heatmap displays a set of data using colored tiles for each variable value within each observation.

Drawing a Heatmap for multivariate data

Use “mtcars” data set available in `"datasets" R package` and draw a heat map using all variables.

```
data(mtcars, package="datasets")
library(superheat)
# scale = TRUE standardizes the variables to a mean of zero and standard deviation of one
superheat(mtcars,
          scale = TRUE,
          left.label.text.size=3,
          bottom.label.text.size=3,
          bottom.label.size = .05,
          row.dendrogram = TRUE )
```



Note that Merc 230 has a high *qsec since the respective colour is* bright yellow, and the Lotus Europa has a low weight since the colour for that cell is dark blue. Also, the Toyota

Corolla and Fiat 128 have similar characteristics. The Lincoln Continental and Cadillac Fleetwood also have similar characteristics.

When using the **superheat** function, the data should be all numeric, and the row names are used to label the left axis. If the desired labels are in a column variable, then the variable must be converted to row names as below:

```
# In plotdata dataframe, if country is a column variable save it as row names
plotdata <- as.data.frame(plotdata)
row.names(plotdata) <- plotdata$country
plotdata$country <- NULL
```

6.3.9 Interactive Graphs

Instead of the static graphs that we discussed earlier, we can also draw interactive graphs through connections with JavaScript libraries, which can be placed on web pages. More details refer htmlwidgets for R.

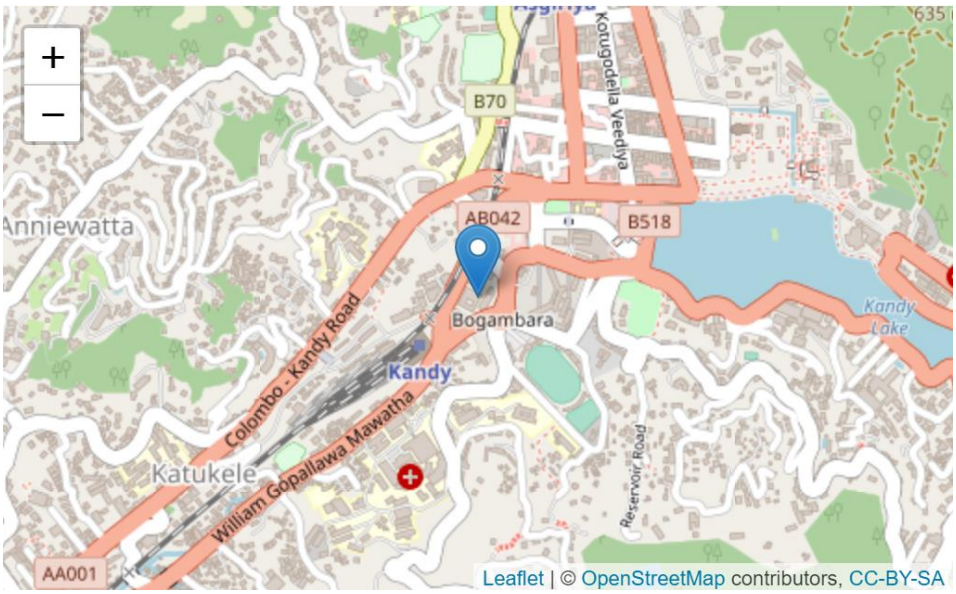
Leaflet Graph

Leaflet is a javascript library for interactive maps. To generate leaflet graphs use the leaflet package in R.

Drawing a Leaflet Graph for multivariate data

Use the following codes to draw a leaflet graph.

```
library(leaflet)
leaflet() %>%
  addTiles() %>%
  addMarkers(lng= 80.6337,
             lat= 7.2906,
             popup="This is my birth place.</br>
             A very beautiful place.")
```



Run the R Code and check the graph. We can zoom in and out with the +/- buttons or mouse wheel.
Refer the following links:
1. https://rstudio.github.io/leaflet/
2. https://www.earthdatascience.org/courses/earth-analytics/get-data-using-apis/leaflet-r/

6.4 Drawing graphs using Plotly

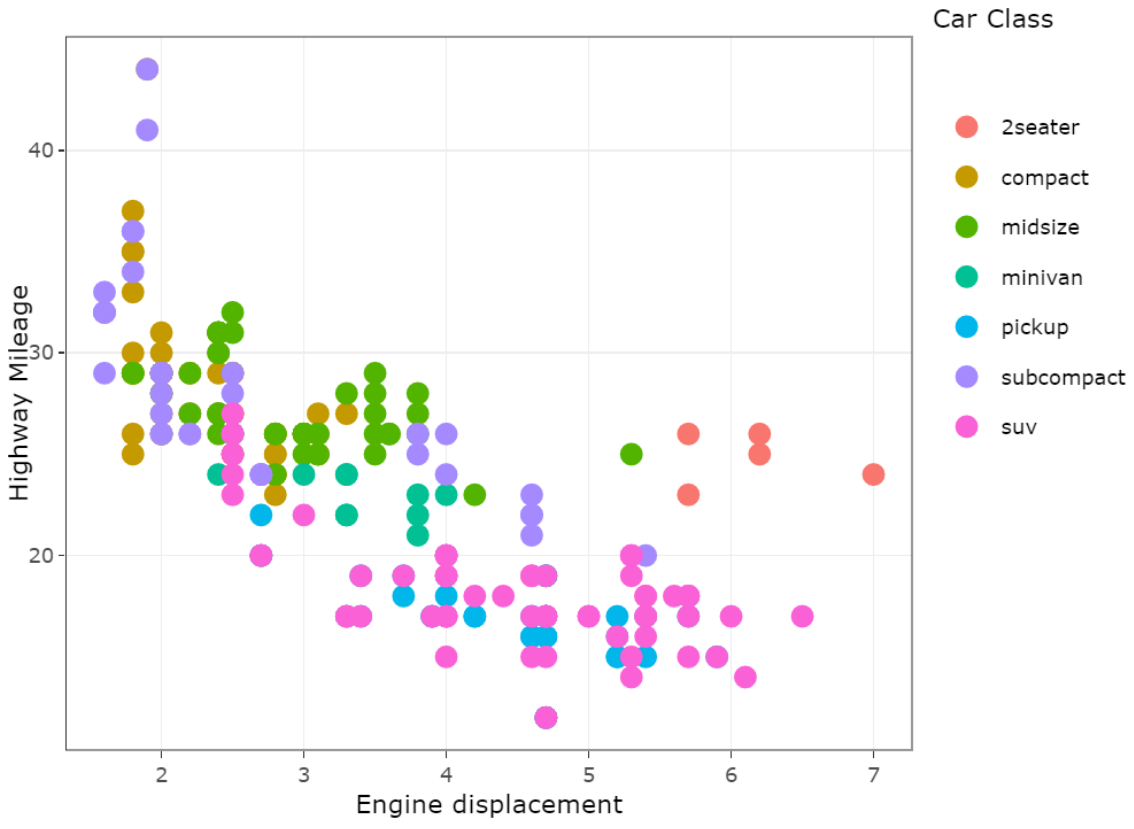
Using Plotly, we can draw high end interactive visualizations.
Again we use the mpg dataset from the ggplot2 package, which contains fuel economy data for 38 popular models of car, for the years 1999 and 2008.

The dataset can be accessed using

```
data(mpg, package="ggplot2")
library(ggplot2)
library(plotly)

p <- ggplot(mpg, aes(x=displ,
                     y=hwy,
                     color=class)) +
 geom_point(size=3) +
 labs(x = "Engine displacement", y = "Highway Mileage", color = "Car
Class") +
  theme_bw()

ggplotly(p)
```



When we place mouse over a point that displays the information about that point. When we click on a legend point, that class of points are removed. Clicking on it again, the respective points return.

6.5 Some other graphing facilities

1. rbokeh is an interface to the Bokeh graphics library.
2. rCharts can create a wide range of interactive graphics.
3. The `highcharter` package provides access to the Highcharts JavaScript graphics library.
4. Choropleth maps using the `choroplethr` package.
   (Refer https://rpubs.com/NateByers/MapsExercises and https://rstudio-pubs-static.s3.amazonaws.com/324400_69a673183ba449e9af4011b1eeb456b9.html
5. Dot density maps to explore spatial relationships using ggmap package. Refer (https://rpubs.com/abhaypadda/plotting-maps-in-R and https://nbisweden.github.io/RaukR-2019/ggmap_Sebastian/lab/ggmap_Sebastian.html)
6. Cook Book for R Graphs: http://www.cookbook-r.com/Graphs/Colors_(ggplot2)/
7. Handling missing values: https://r4ds.hadley.nz/missing-values.html
8. Pivot data: https://tidyr.tidyverse.org/reference/pivot_longer.html
9. Tidy data: https://tidyr.tidyverse.org/articles/tidy-data.html

References

1. A ggplot2 grammar guide: https://evamaerey.github.io/ggplot2_grammar_guide/ggplot2_grammar_guide#1
2. An Introduction to R for Research: https://bookdown.org/rwnahhas/IntroToR/
3. Barplot with error bars: https://r-graph-gallery.com/4-barplot-with-error-bar.html
4. Creating graphs online: https://rawgraphs.io/learning
5. Dash R user guide for plotly: https://dashr.plotly.com/
6. Datavis with R: https://rkabacoff.github.io/datavis/Bivariate.html
7. Getting started with Plotly in R: https://plotly.com/r/getting-started/#getting-started-with-plotly-for-r
8. ggplot2 and extensions geoms: https://evamaerey.github.io/ggplot2_grammar_guide/geoms.html#1
9. Plotly R graphing library: https://plotly.com/r/
10. R Charts: https://r-charts.com/ggplot2/
11. R Markdown Cookbook: https://bookdown.org/yihui/rmarkdown-cookbook/chunk-options.html
12. R Markdown from R Studio: https://rmarkdown.rstudio.com/lesson-3.html
13. STHDA Publication ready graphs: http://www.sthda.com/english/articles/24-ggpubr-publication-ready-plots/79-plot-meansmedians-and-error-bars/
14. The Magic of ggplot2: https://r-bootcamp.netlify.app/chapter1